# 모찌 (Mochi) 앱 개발 최종 보고서

**개발자**: 개인 개발
**개발 기간**: 2025-11-13 ~ 2025-11-19
**플랫폼**: Flutter (Android/iOS)
**제출일**: 2025-11-19

---

## 목차

1. 프로젝트 개요
2. 기획 배경 및 목표
3. 화면 구상도
4. 화면 연결 흐름도
5. 화면 기능 명세 및 화면 구성 설명
6. 핵심 코드
7. 개인 구현 부분
8. 차별점
9. 향후 개선 방향 및 느낀점

---

## 1. 프로젝트 개요

**모찌 (Mochi)**는 실시간 위치 기반 소셜 앱으로, 친구들과 현재 위치를 공유하고 AI가 추론한 상황별 상태 메시지를 통해 소통할 수 있는 서비스입니다.

### 1.1 핵심 기능

- **실시간 위치 공유**: GPS를 활용한 실시간 위치 추적 및 Google Maps를 통한 지도 표시
- **AI 기반 상태 추론**: Google Gemini AI를 활용한 위치와 시간 기반 지능형 상태 메시지 생성
- **실시간 채팅**: Firestore를 활용한 1:1 실시간 채팅 기능
- **친구 관리**: 연락처 기반 친구 찾기 및 친구 요청/수락 기능
- **소셜 로그인**: Google 및 Kakao 소셜 로그인 지원

### 1.2 기술 스택

- **프레임워크**: Flutter 3.9.2+
- **백엔드**: Firebase (Authentication, Firestore, Cloud Messaging)
- **지도**: Google Maps Flutter
- **AI**: Google Gemini Pro
- **상태 관리**: Provider 패턴
- **언어**: Dart

---

## 2. 기획 배경 및 목표

### 2.1 기획 배경

기존 위치 공유 앱들은 단순히 위치만 표시하거나, 사용자가 직접 상태를 입력해야 하는 불편함이 있었습니다. 또한 친구들의 현재 상황을 파악하기 어려워 소통에 어려움이 있었습니다.

**모찌**는 이러한 문제점을 해결하기 위해 다음과 같은 기획 목표를 가지고 개발되었습니다:

1. **AI 기반 자동 상태 추론**: 사용자의 위치와 시간 정보를 바탕으로 AI가 자동으로 상황을 추론하여 상태 메시지를 생성
2. **실시간 위치 공유**: 친구들의 실시간 위치를 지도에서 한눈에 확인
3. **직관적인 UI/UX**: 카카오톡 스타일의 세련된 채팅 UI와 Material Design 3 기반의 현대적 디자인
4. **간편한 소셜 로그인**: Google, Kakao 소셜 로그인을 통한 빠른 접근

### 2.2 목표

- 사용자의 위치와 시간을 종합적으로 분석하여 적절한 상태 메시지를 자동으로 추론
- 친구들의 실시간 위치를 지도에서 직관적으로 확인
- 친구와의 소통을 위한 실시간 채팅 기능 제공
- 사용자 친화적인 UI/UX로 앱 사용성 향상

---

## 3. 화면 구상도

다음은 모찌 앱의 전체 화면 구상도입니다.

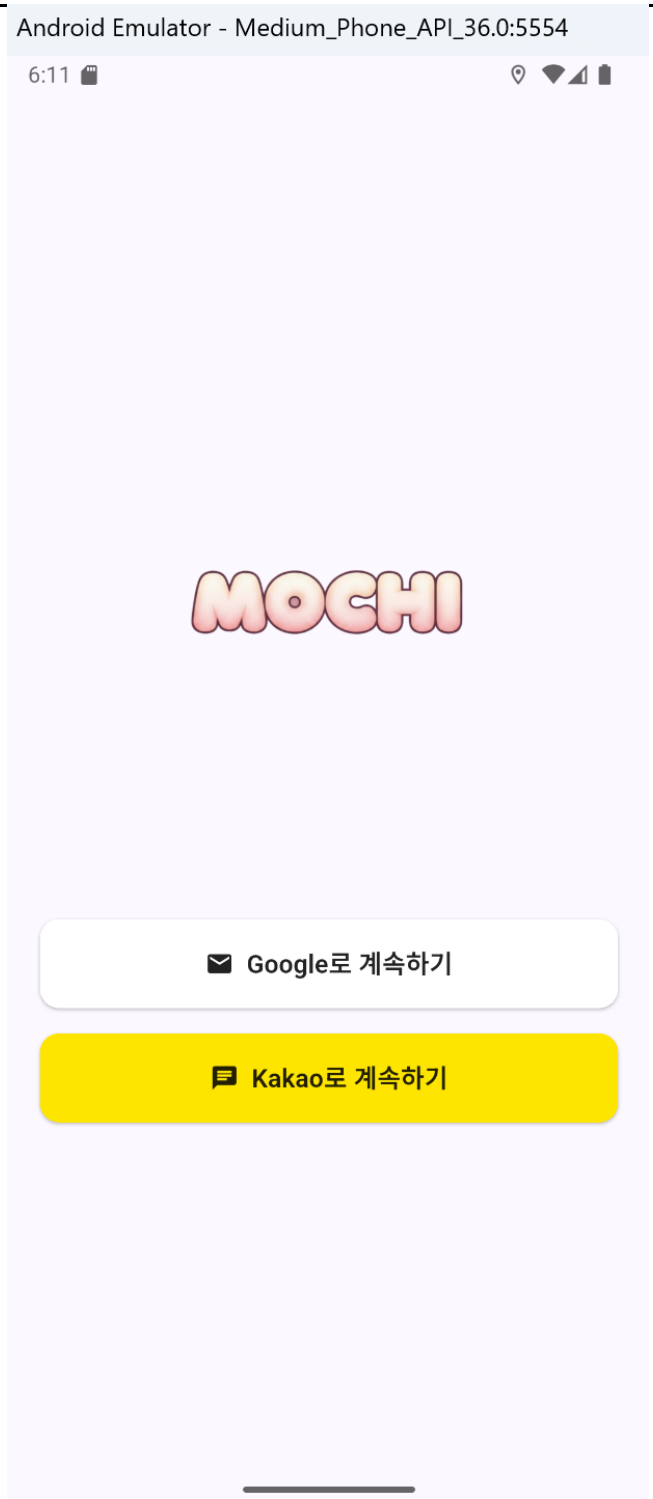
### 3.1 주요 화면 목록

1. **스플래시 스크린**: 앱 로고 표시 및 초기화
2. **로그인 화면**: 소셜 로그인 (Google, Kakao)
3. **지도 화면**: Google Maps + 실시간 위치 마커
4. **채팅 목록 화면**: 참여 중인 채팅방 목록
5. **채팅 화면**: 1:1 실시간 채팅
6. **친구 목록 화면**: 추가한 친구 목록
7. **마이페이지**: 프로필 및 설정

### 3.2 화면 구상도 스크린샷

다음은 모찌 앱의 주요 화면 스크린샷입니다:

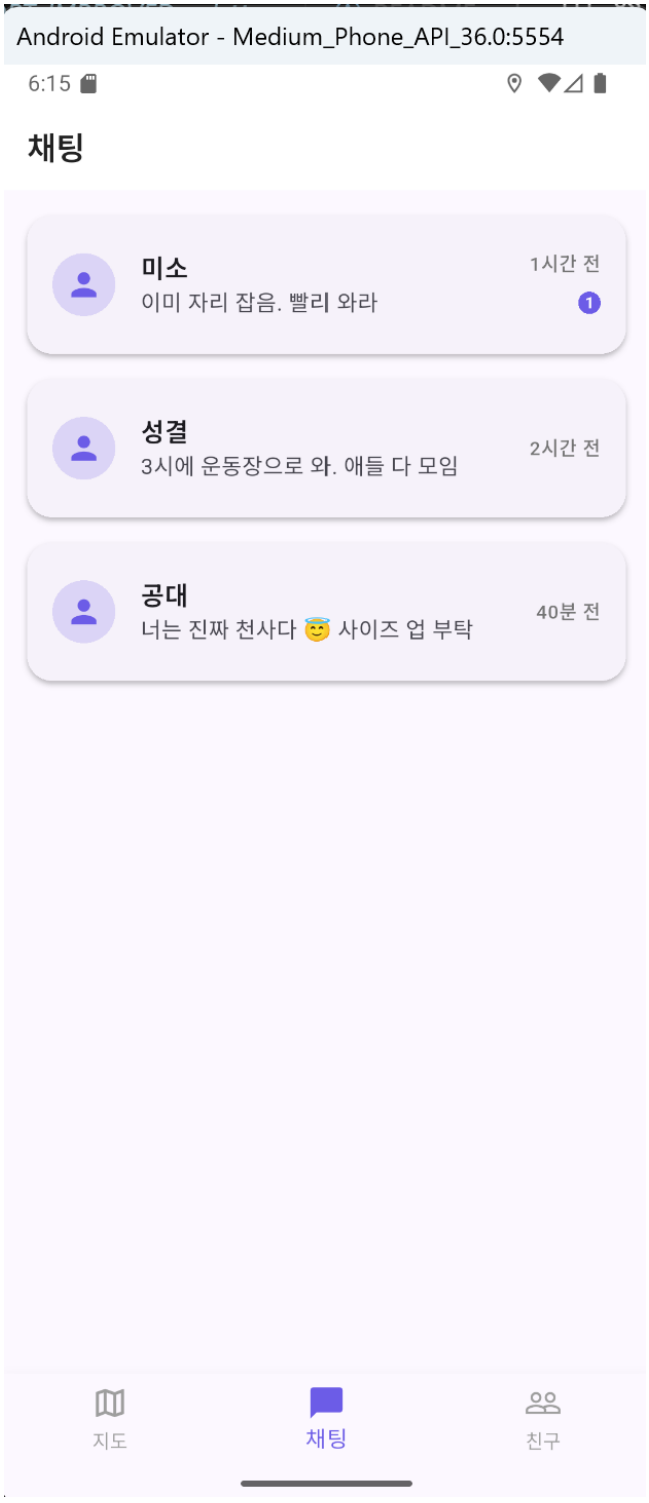
| 스플래시 스크린 | 로그인 화면 |
|---|---|

| 스플래시 스크린 | 로그인 화면 |
| --- | --- |
|  |  |
| 지도 화면 | 채팅 목록 화면 |

**스플래시 스크린**




**로그인 화면**

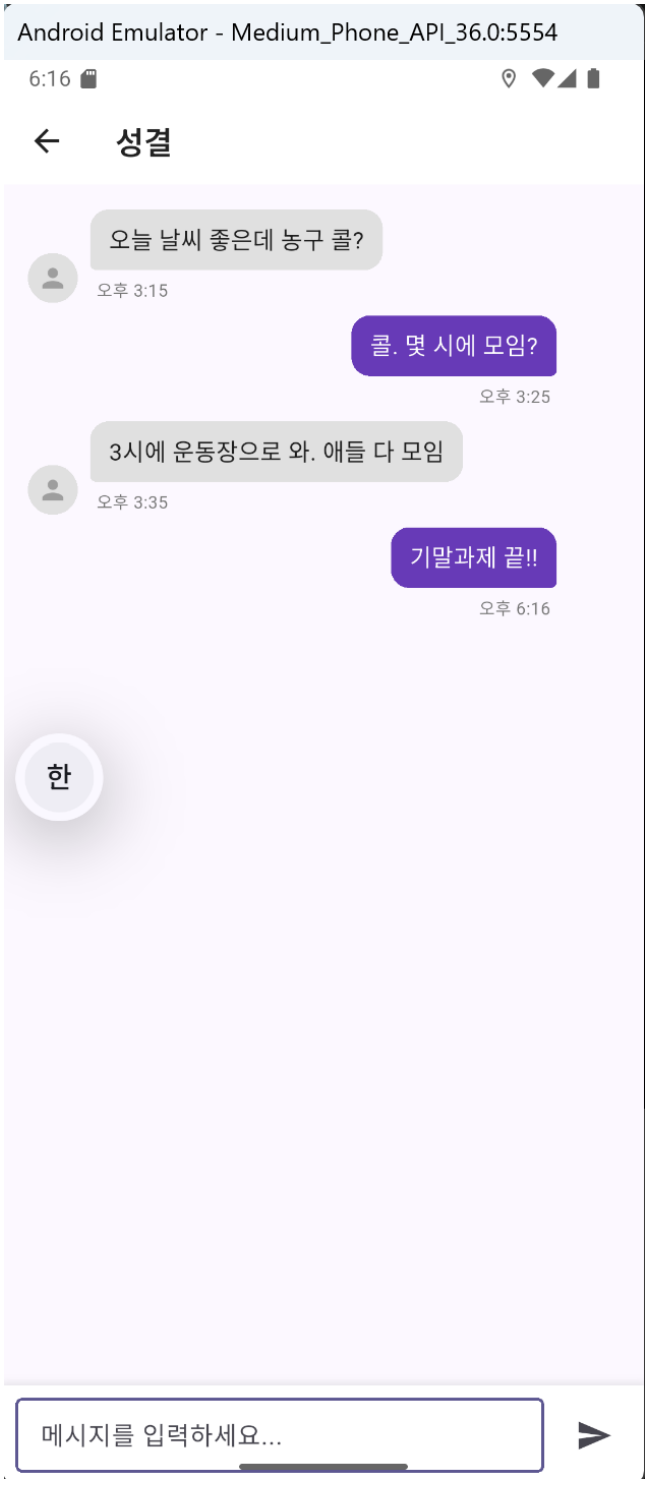


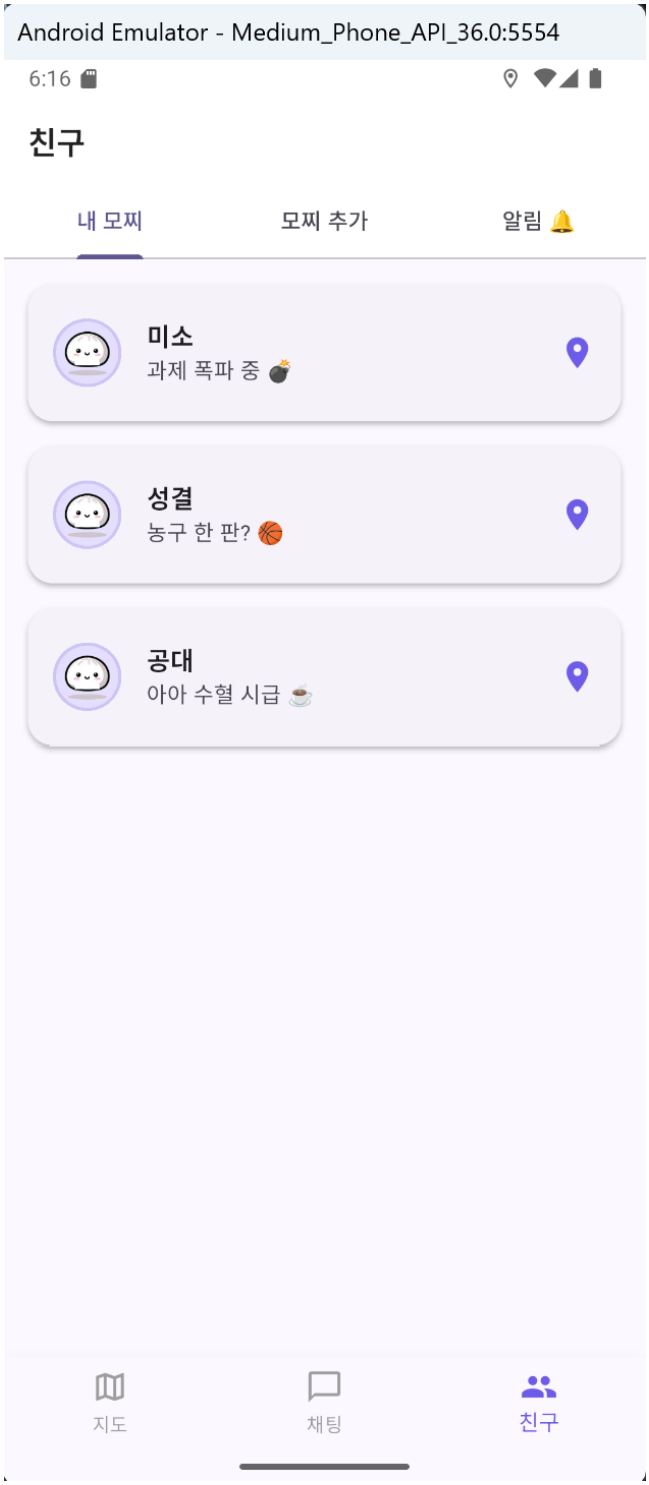

**채팅 화면**

**친구 목록 화면**

**스플래시 스크린**

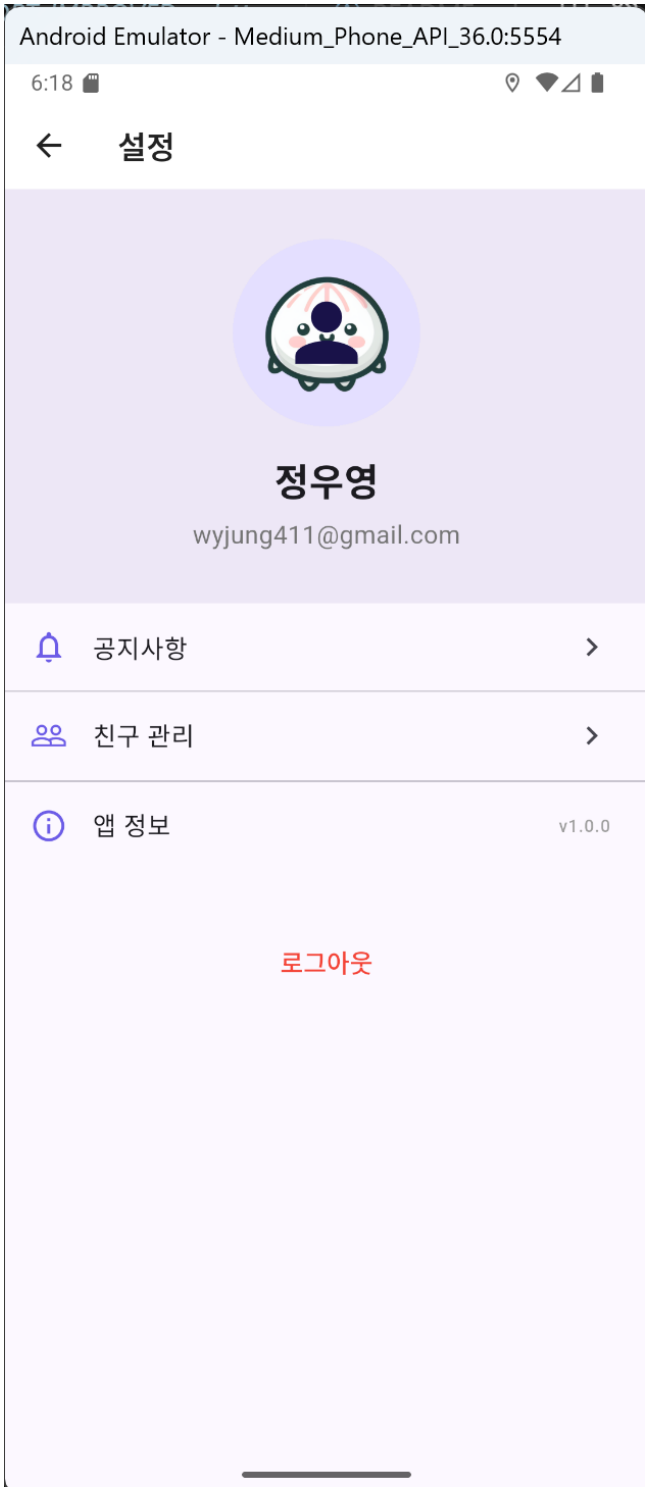
**로그인 화면**




**마이페이지**

| 스플래시 스크린 | 로그인 화면 |
| --- | --- |




## 4. 화면 연결 흐름도

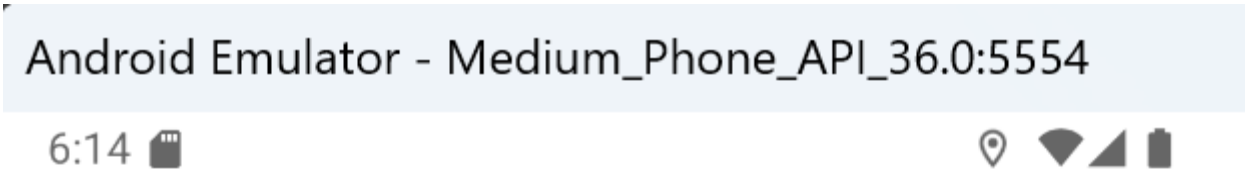
```
앱 실행
  ↓
[스플래시 스크린] (2초, 페이드 인/아웃 애니메이션)
  ↓
[인증 게이트]
  ├─ 로그인 안됨 → [로그인 화면]
  │                    ├─ Google 로그인 버튼
```

```
    │                    └ Kakao 로그인 버튼
    │                    ↓ (로그인 성공)
    │                    [메인 네비게이터]
    │
    └ 로그인 됨 → [메인 네비게이터]
                    ├ [지도 화면] (Tab 1) ← 기본 화면
                    │     ├ Google Maps
                    │     ├ 내 위치 마커 (모찌 이미지)
                    │     ├ 친구 위치 마커들 (친구 마커 이미지)
                    │     ├ AI 추천 버튼 (우측 하단)
                    │     ├ 상태 변경 버튼 (우측 하단)
                    │     ├ 내 위치로 이동 버튼 (좌측 하단)
                    │     └ 프로필 버튼 (우측 상단) → [마이페이지]
                    │
                    ├ [채팅 목록 화면] (Tab 2)
                    │     └ 채팅방 카드 클릭
                    │           ↓
                    │           [채팅 화면]
                    │                 ├ 메시지 목록 (StreamBuilder)
                    │                 └ 메시지 입력창
                    │
                    └ [친구 목록 화면] (Tab 3)
                          ├ 내 모찌 탭
                          ├ 모찌 추가 탭
                          └ 알림 탭
                          ↓ (친구 카드 클릭)
                          [채팅 화면]
```

---

# 5. 화면 기능 명세 및 화면 구성 설명

## 5.1 스플래시 스크린 (SplashScreen)




MOCHI

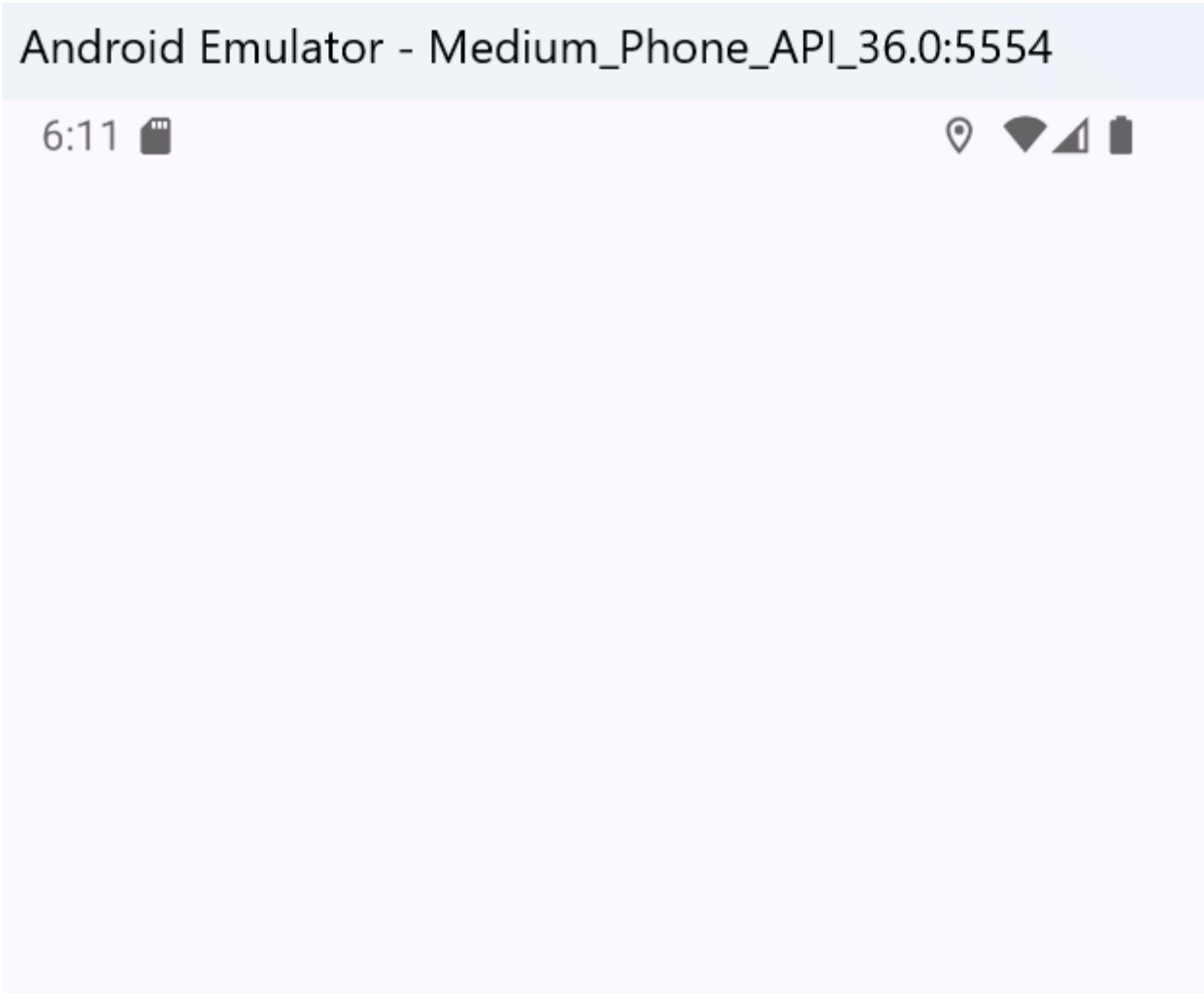
**기능 명세:**

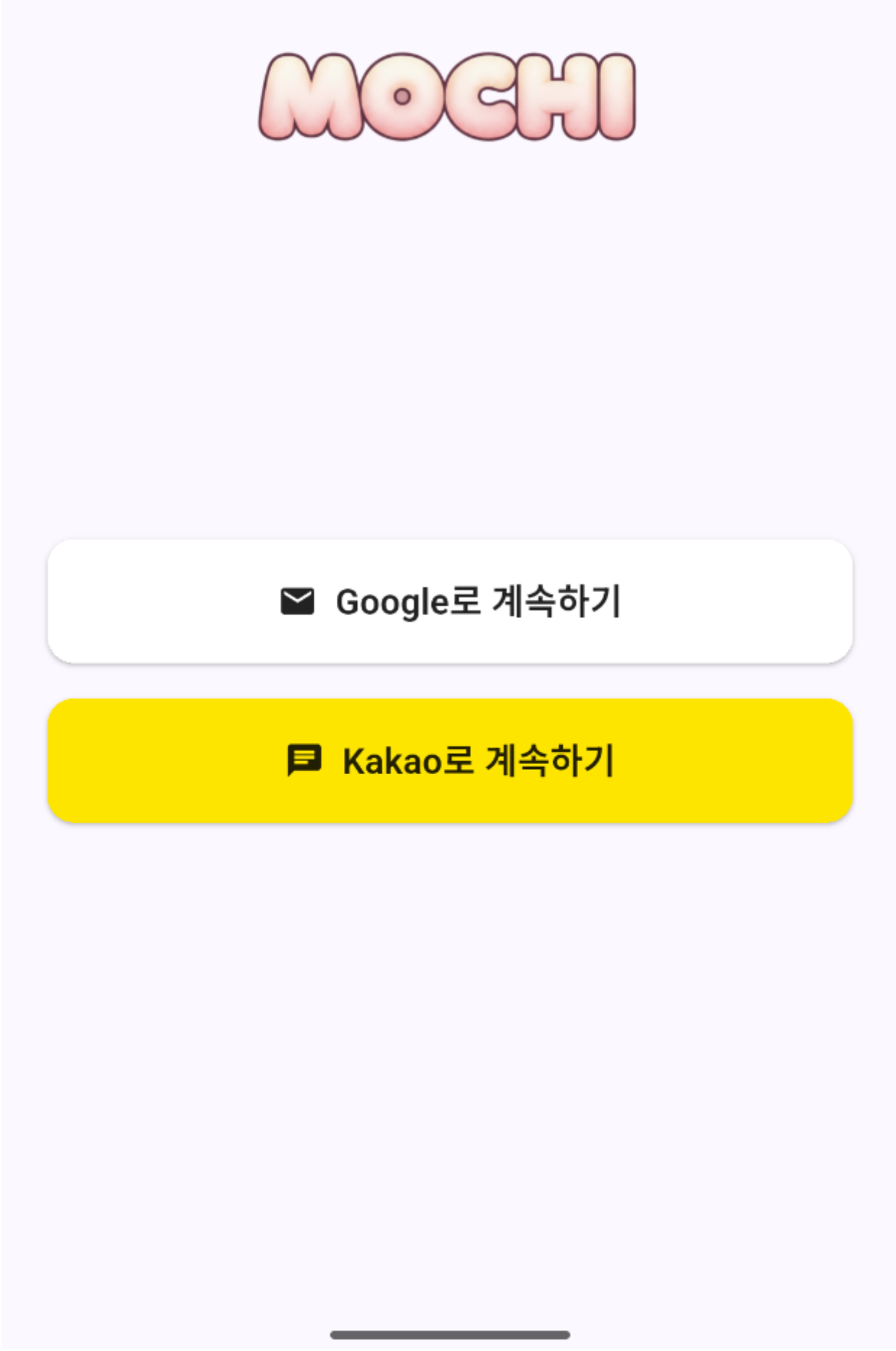
- 앱 실행 시 첫 화면으로 표시
- 앱 로고 이미지를 페이드 인/아웃 애니메이션과 함께 표시
- 2초 후 자동으로 `AuthGate`로 이동

**구성 요소:**

| 위젯 | 명칭 | 설명 |
|---|---|---|
| `Scaffold` | 메인 컨테이너 | 흰색 배경의 전체 화면 구조 |
| `FadeTransition` | 페이드 애니메이션 | 로고 이미지의 페이드 인/아웃 효과 |
| `Image.asset` | 앱 로고 이미지 | `splash_logo.png` 이미지 표시 (width: 200, height: 200) |
| `AnimationController` | 애니메이션 컨트롤러 | 2초 동안 애니메이션 제어 |
| `Timer` | 자동 이동 타이머 | 2초 후 `AuthGate`로 자동 이동 |

5.2 로그인 화면 (LoginScreen)






**기능 명세:**

- 소셜 로그인 (Google, Kakao)을 통한 인증

- 로그인 성공 시 `MainNavigator`로 이동
- 로그인 실패 시 SnackBar로 에러 메시지 표시

**구성 요소:**

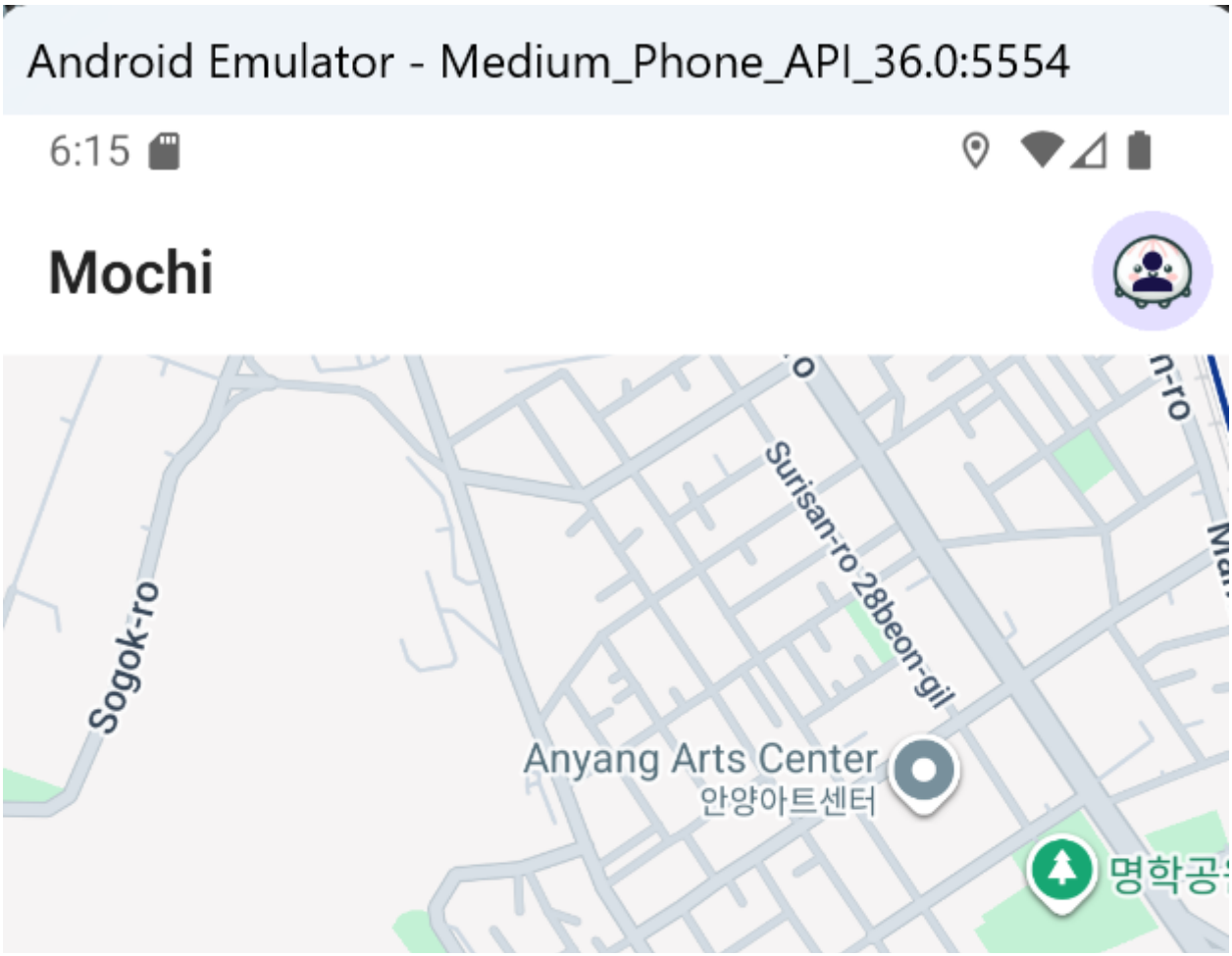
| 위젯 | 명칭 | 설명 |
|---|---|---|
| `Image.asset` | 앱 로고 | `splash_logo.png` 이미지 표시 (width: 200, height: 200) |
| `SocialLoginButton` | Google 로그인 버튼 | 흰색 배경, 검은색 텍스트, 이메일 아이콘 |
| `SocialLoginButton` | Kakao 로그인 버튼 | 노란색 배경 (#FEE500), 검은색 텍스트, 채팅 아이콘 |
| `CircularProgressIndicator` | 로딩 인디케이터 | 로그인 시도 중 표시 |
| `SnackBar` | 에러 메시지 | 로그인 실패 시 표시 |

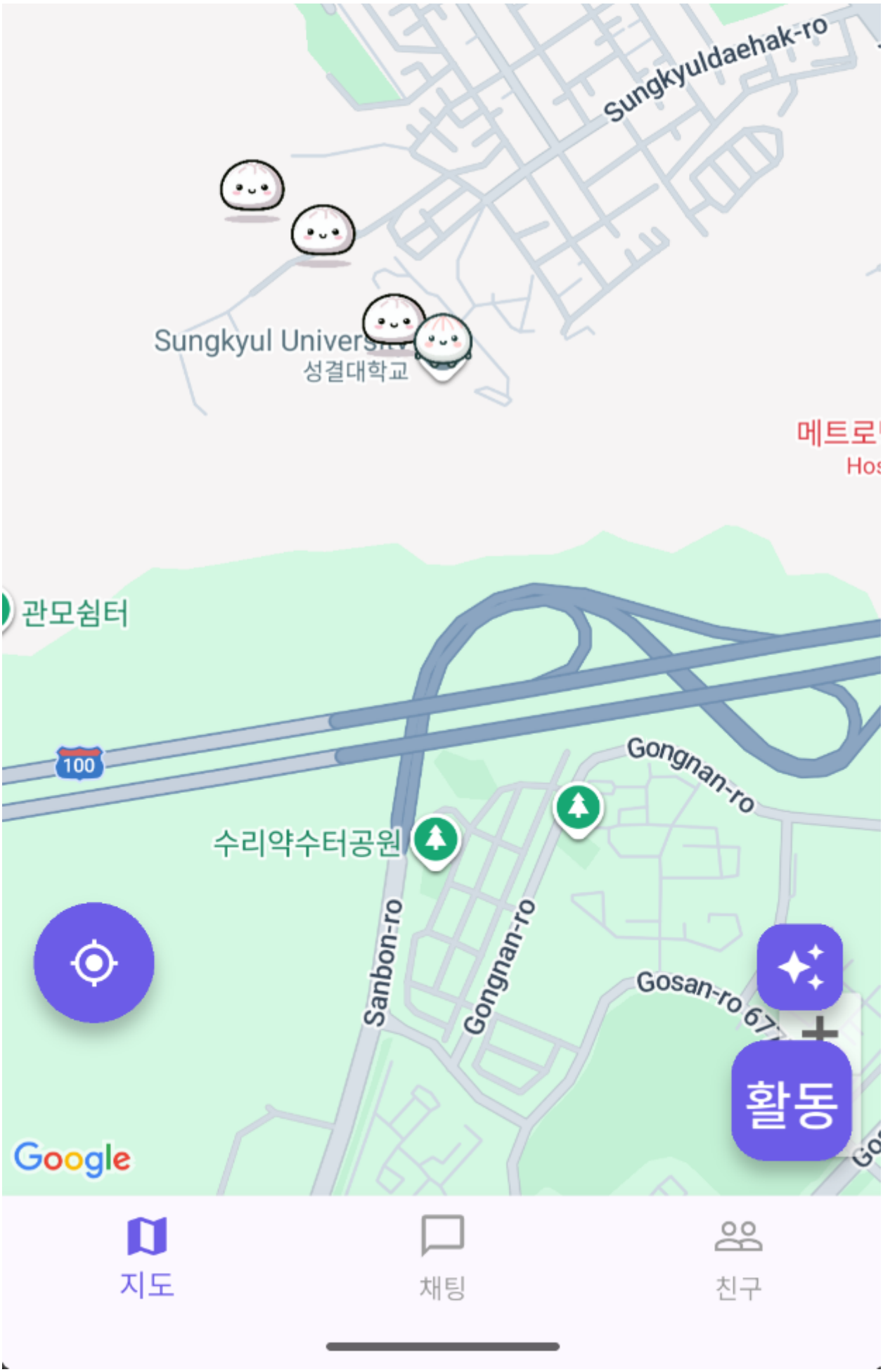
**Firebase 연동 기능:**

- `AuthService.signInWithGoogle()`: Google 로그인 실행
- `AuthService.signInWithKakao()`: Kakao 로그인 실행
- 로그인 성공 시 Firestore에 사용자 정보 자동 저장

---

5.3 지도 화면 (MapScreen)




Sungkyuldaehak-ro
Sungkyul University
성결대학교
관모쉼터
100
Gongnan-ro
수리약수터공원
Sanbon-ro
Gongnan-ro
Gosan-ro
활동
Google
지도
채팅
친구

**기능 명세:**

- Google Maps를 사용한 지도 표시
- 내 위치 및 친구들의 실시간 위치를 커스텀 마커로 표시
- AI 기반 상태 추천 및 수동 상태 변경 기능
- 내 위치로 빠르게 이동하는 기능

**구성 요소:**

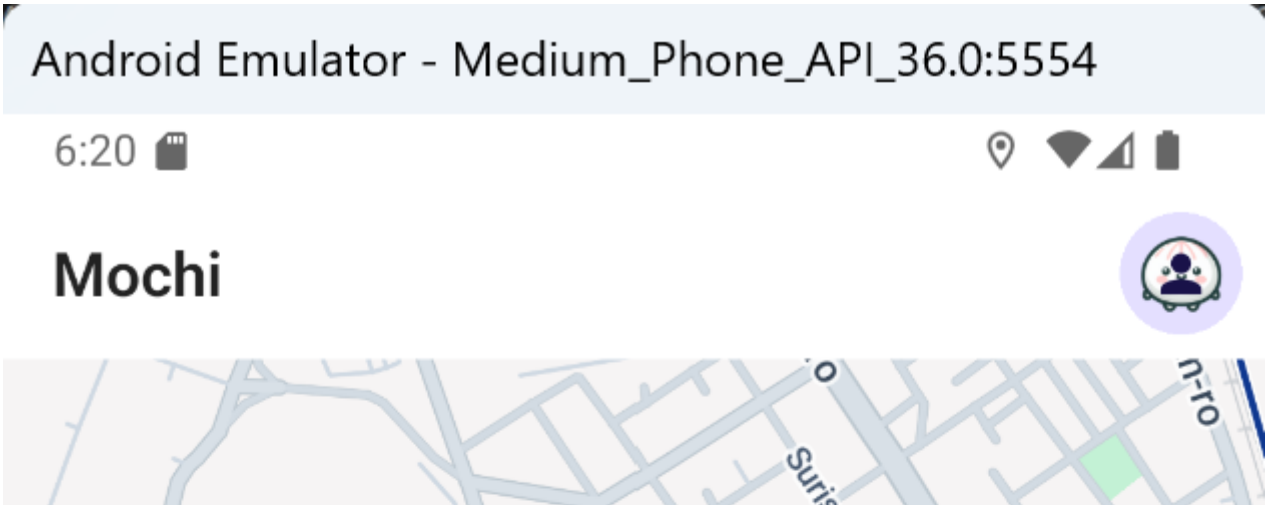
| 위젯 | 명칭 | 설명 |
|---|---|---|
| `GoogleMap` | 지도 위젯 | Google Maps 지도 표시, 줌, 패닝 등 기본 기능 |
| `Marker` | 내 위치 마커 | `marker_mochi.png` 이미지 사용, 현재 상태 메시지 표시 |
| `Marker` | 친구 위치 마커 | `marker_friend.png` 이미지 사용, 친구 이름 및 상태 표시 |
| `InfoWindow` | 마커 정보창 | 마커 클릭 시 친구 이름과 상태 메시지 표시 |
| `FloatingActionButton.small` | AI 추천 버튼 | 우측 하단, 클릭 시 AI가 상태 추천 |
| `FloatingActionButton` | 상태 변경 버튼 | 우측 하단, 클릭 시 상태 선택 다이얼로그 표시 |
| `FloatingActionButton` | 내 위치로 이동 버튼 | 좌측 하단, 클릭 시 현재 위치로 카메라 이동 |
| `CircleAvatar` | 프로필 버튼 | 우측 상단 AppBar, 클릭 시 마이페이지로 이동 |
| `Positioned` | 버튼 위치 지정 | Stack 내에서 버튼 위치 조정 |

**Firebase 연동 기능:**

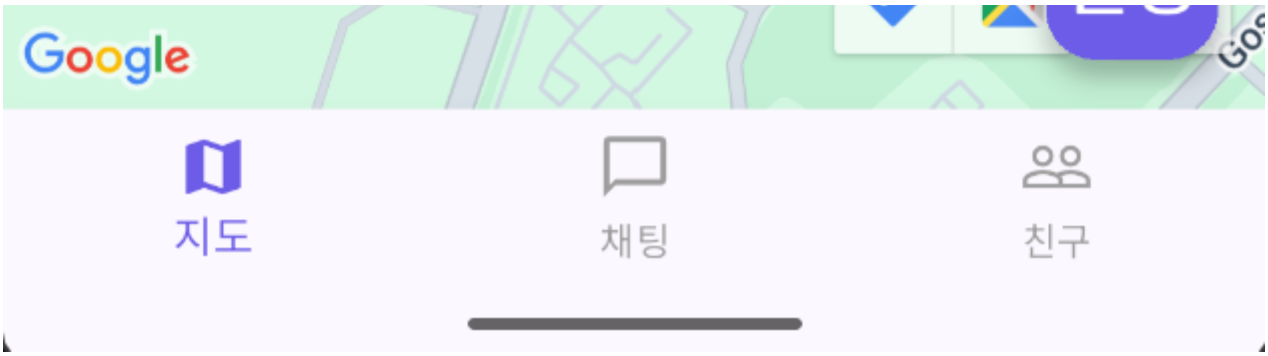
- `DatabaseService.getMyFriendsStream()`: 친구 목록 실시간 구독
- `DatabaseService.getFriendsLocationsStream()`: 친구 위치 실시간 구독
- `LocationProvider`: GPS 위치 추적 및 Firestore 업데이트

**특별 기능:**

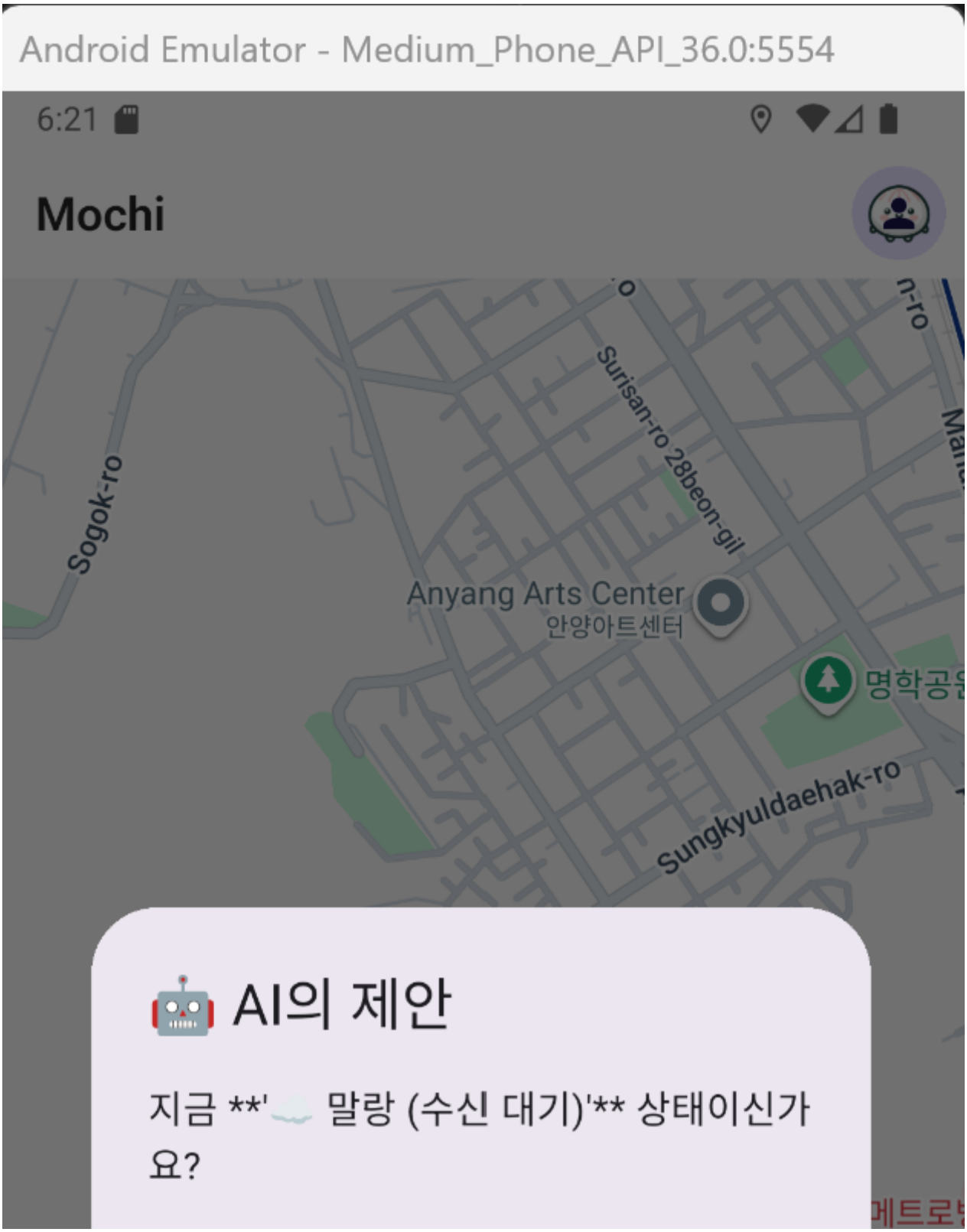
- 초기 위치 로드 시 자동으로 현재 위치로 카메라 이동
- 10미터 이상 이동 시에만 위치 업데이트 (배터리 최적화)
- 마커 클릭 시 InfoWindow에 친구 이름과 상태 메시지 표시

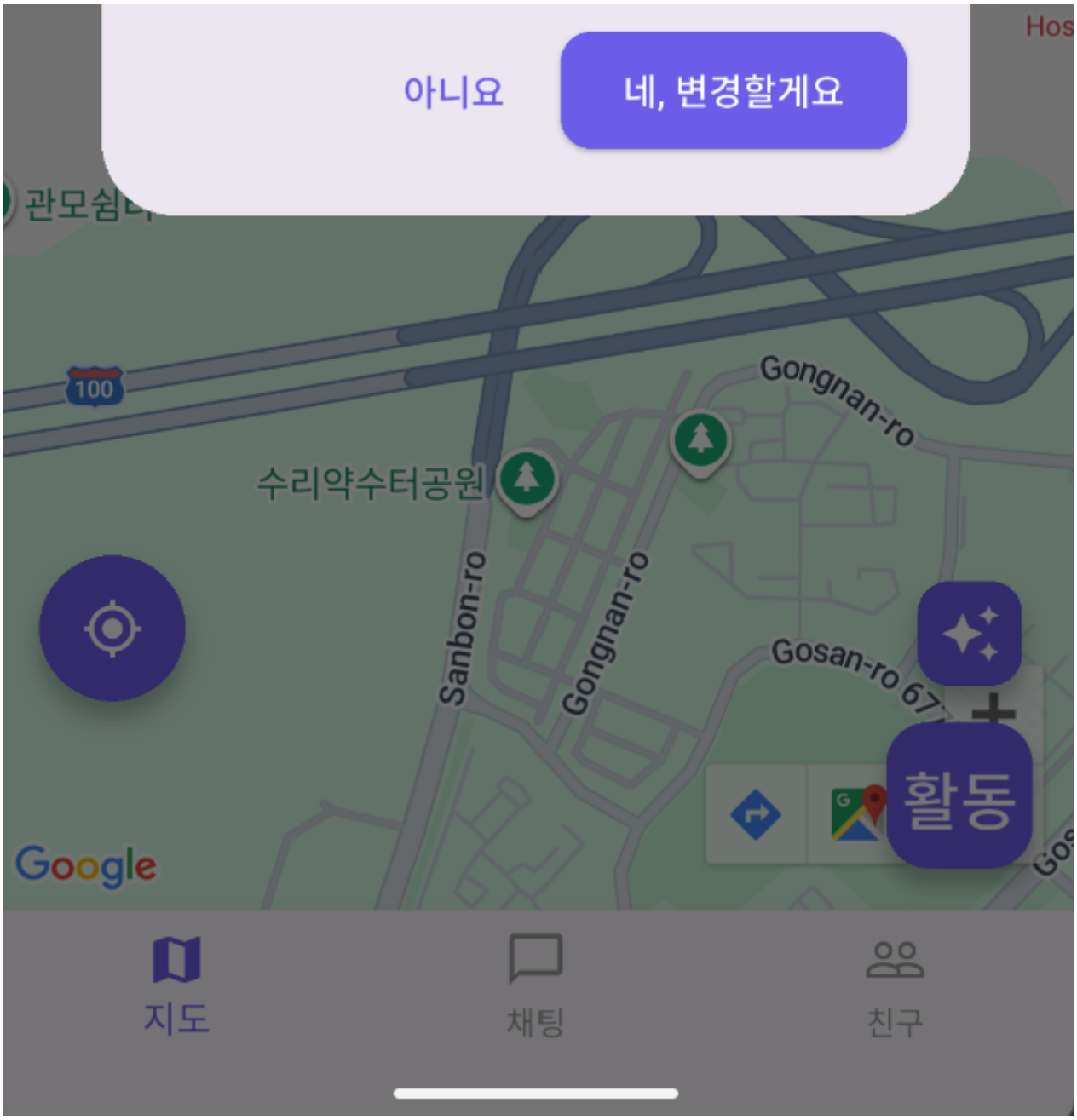



Sogok-ro
Anyang Arts Center
안양아트센터
명학공원
Sungkyuldaehak-ro
내 위치
활동 중
Sungkyul University
성결대학교
관모쉼터
100
수리약수터공원
Gongnan-ro
Sanbon-ro
Gongnan-ro
Gosan-ro

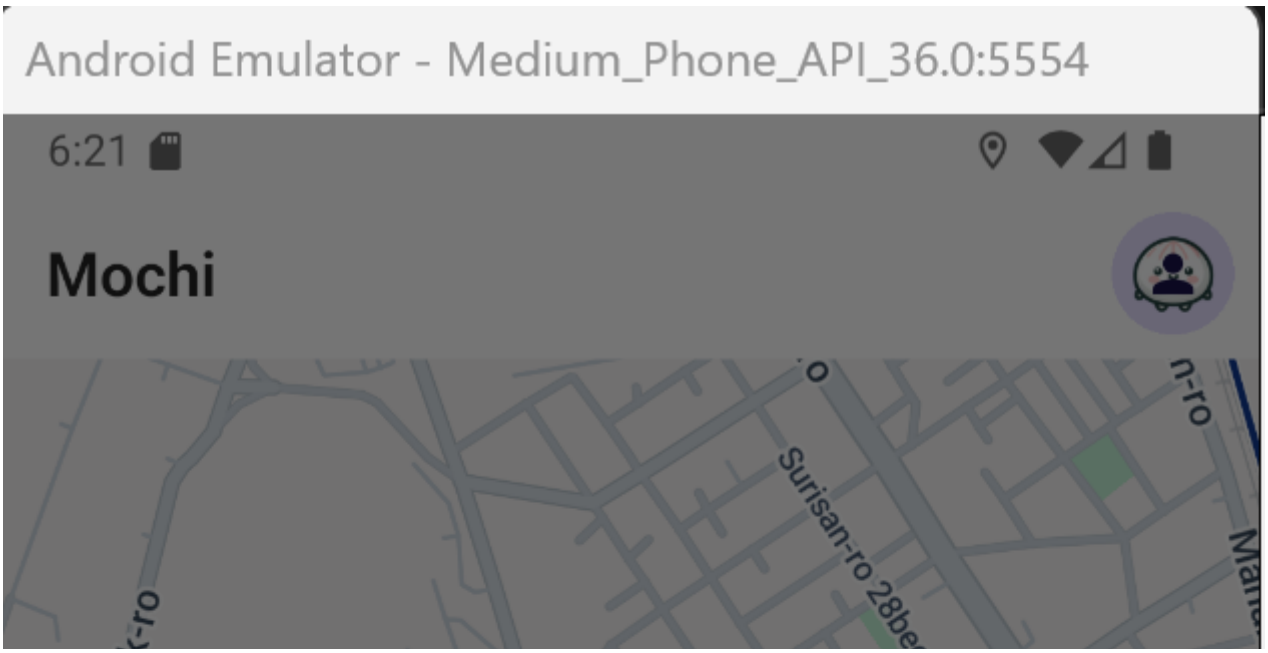


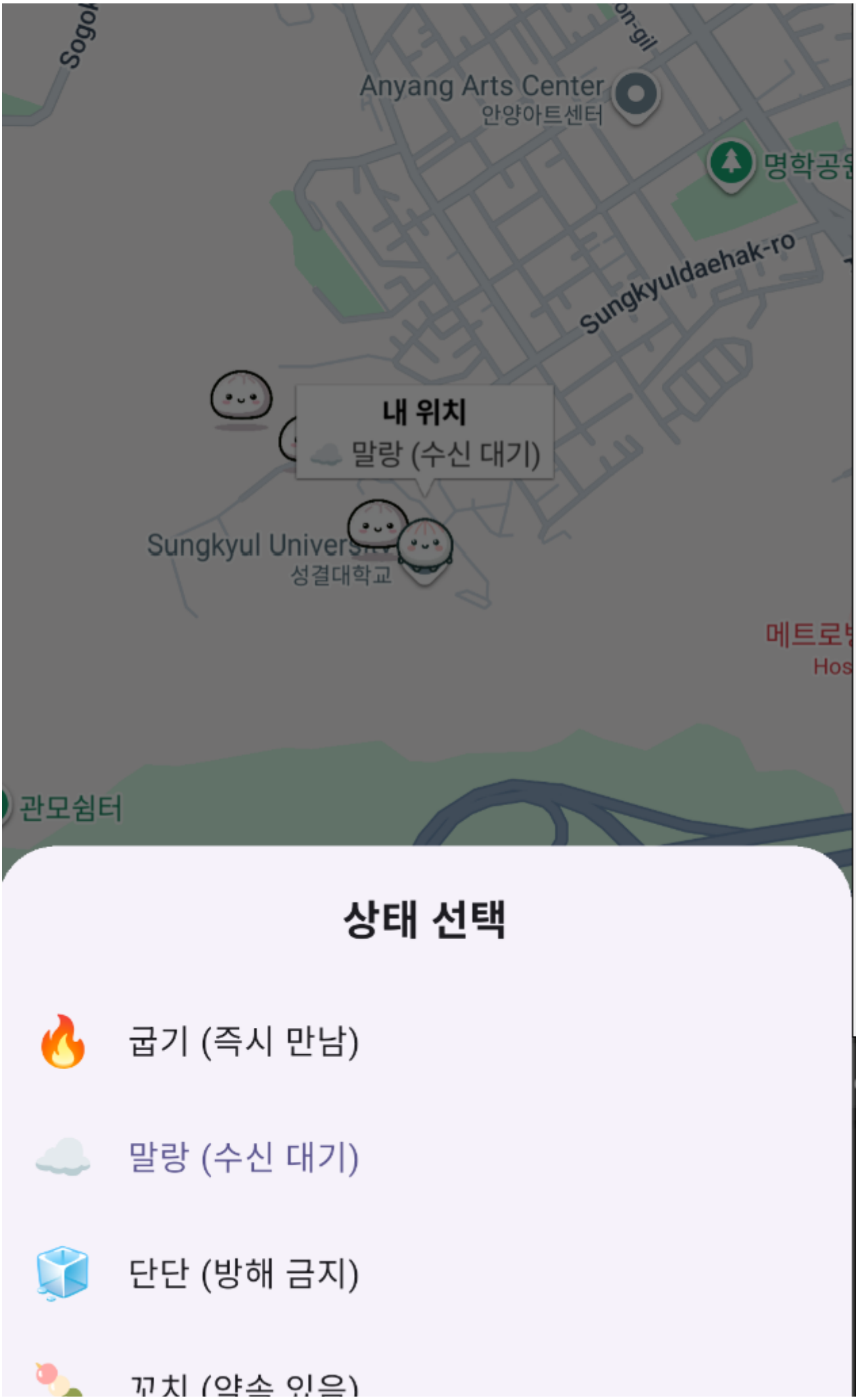

**AI 상태 추천 다이얼로그:**




아니요
네, 변경할게요
Hos
관모쉼터
100
Gongnan-ro
수리약수터공원
Sanbon-ro
Gongnan-ro
Gosan-ro 67
활동
Google
지도
채팅
친구

**상태 변경 다이얼로그:**

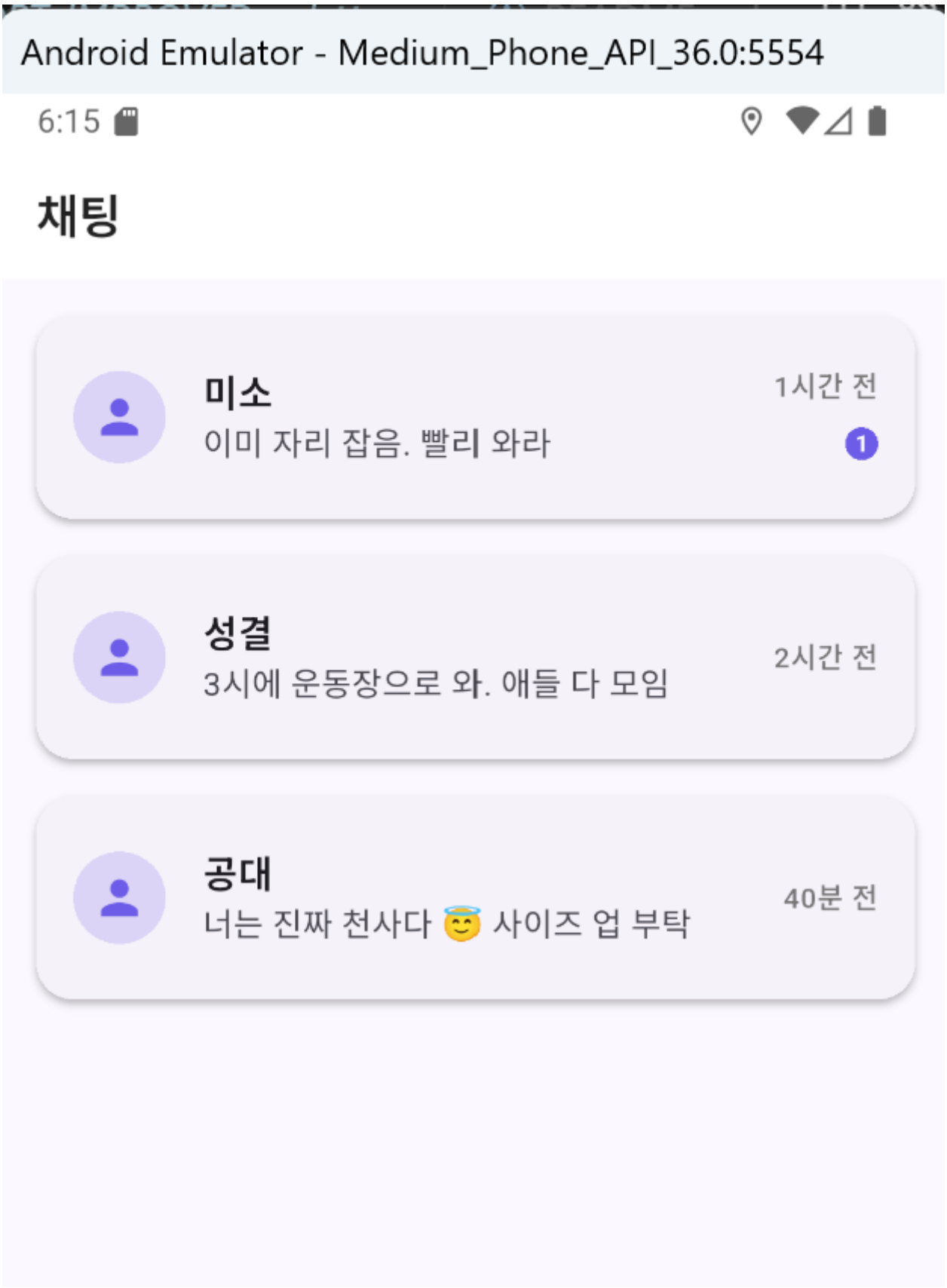

Android Emulator - Medium_Phone_API_36.0:5554
6:21
Mochi
Surisan-ro 28be

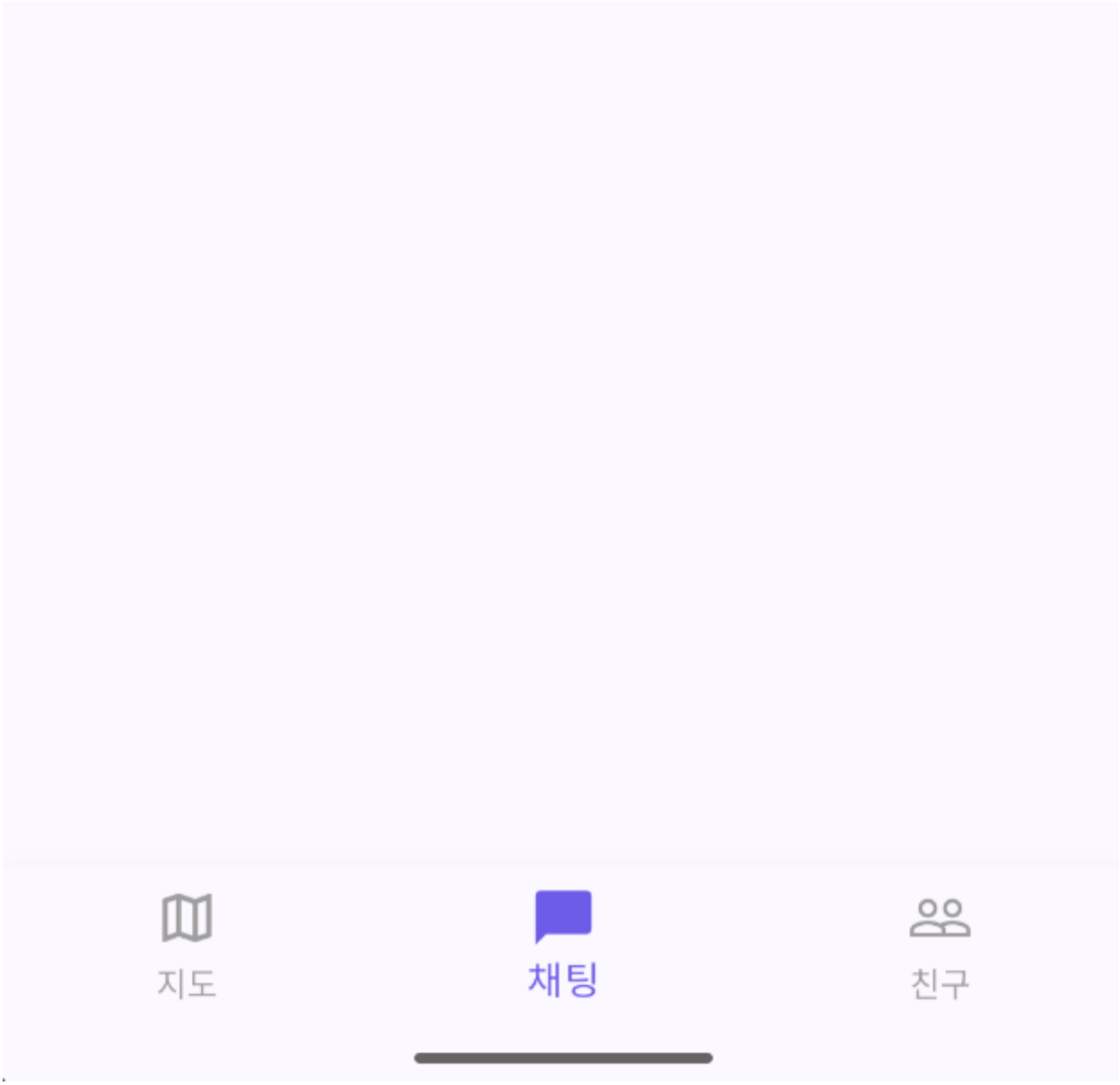

Anyang Arts Center
안양아트센터
명학공원
Sungkyuldaehak-ro
내 위치
말랑 (수신 대기)
Sungkyul University
성결대학교
관모쉼터
상태 선택
굽기 (즉시 만남)
말랑 (수신 대기)
단단 (방해 금지)



---

5.4 채팅 목록 화면 (ChatListScreen)






**기능 명세:**

- 참여 중인 모든 채팅방 목록을 카드 형태로 표시
- 각 채팅방의 마지막 메시지와 시간 표시
- 읽지 않은 메시지 배지 표시
- 채팅방 클릭 시 해당 채팅 화면으로 이동

**구성 요소:**

| 위젯 | 명칭 | 설명 |
|---|---|---|
| `ListView.builder` | 채팅방 리스트 | 채팅방 목록을 동적으로 생성 |
| `Card` | 채팅방 카드 | 둥근 모서리 (borderRadius: 16), 여백 적용 |
| `ListTile` | 채팅방 아이템 | 프로필 아이콘, 이름, 메시지, 시간 포함 |
| `CircleAvatar` | 프로필 아이콘 | 보라색 반투명 배경, 흰색 사람 아이콘 |
| `Text` | 친구 이름 | 굵은 글씨 (fontWeight: w600) |
| `Text` | 마지막 메시지 | 최대 1줄, 말줄임표 처리 |

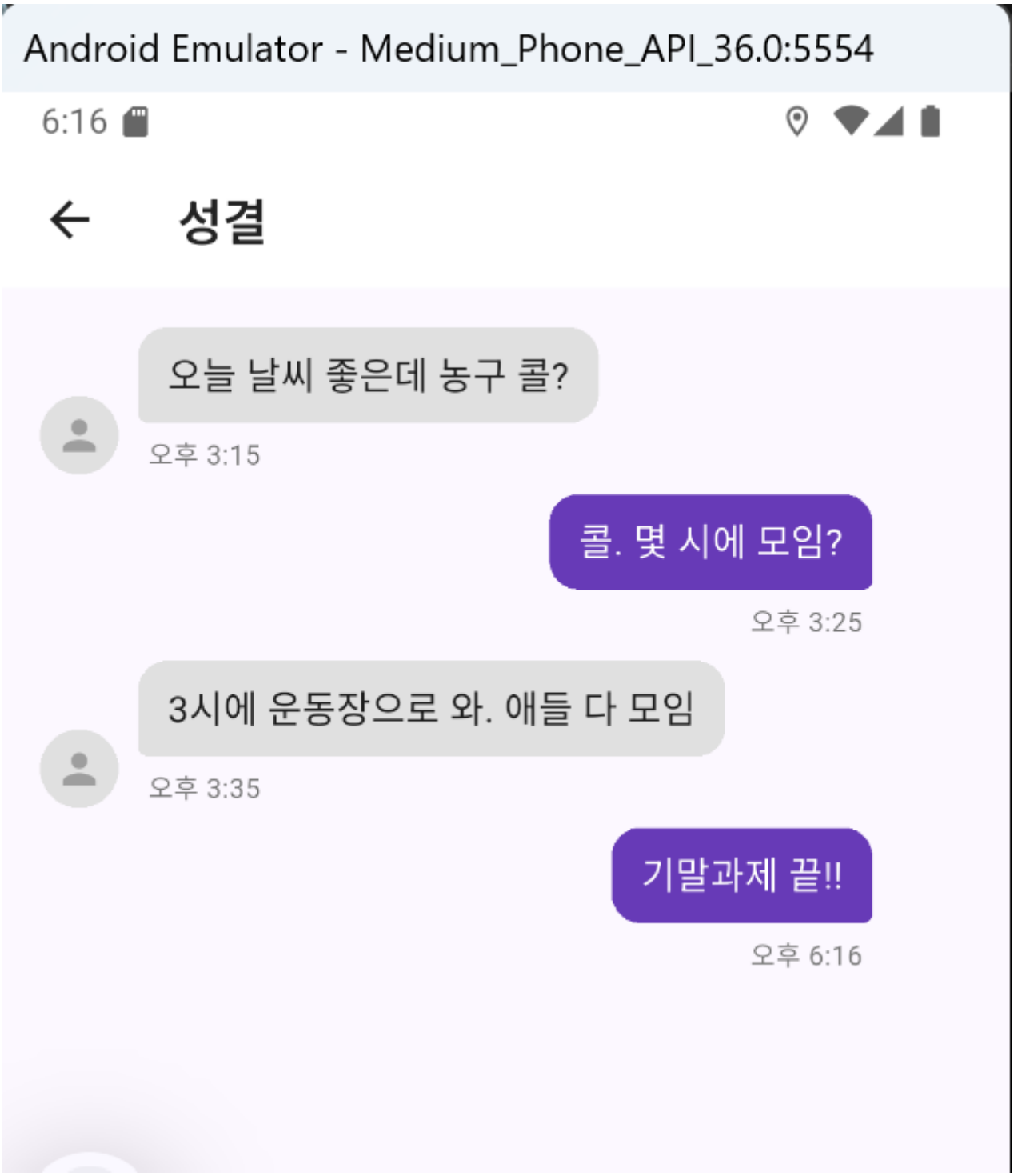
| 위젯 | 명칭 | 설명 |
|---|---|---|
| `Text` | 메시지 시간 | 작은 회색 폰트 (fontSize: 12) |
| `Container` | 읽지 않은 메시지 배지 | 보라색 원형 배지, 흰색 숫자 |
| `EmptyStateWidget` | 빈 상태 위젯 | 채팅방이 없을 때 안내 메시지 표시 |

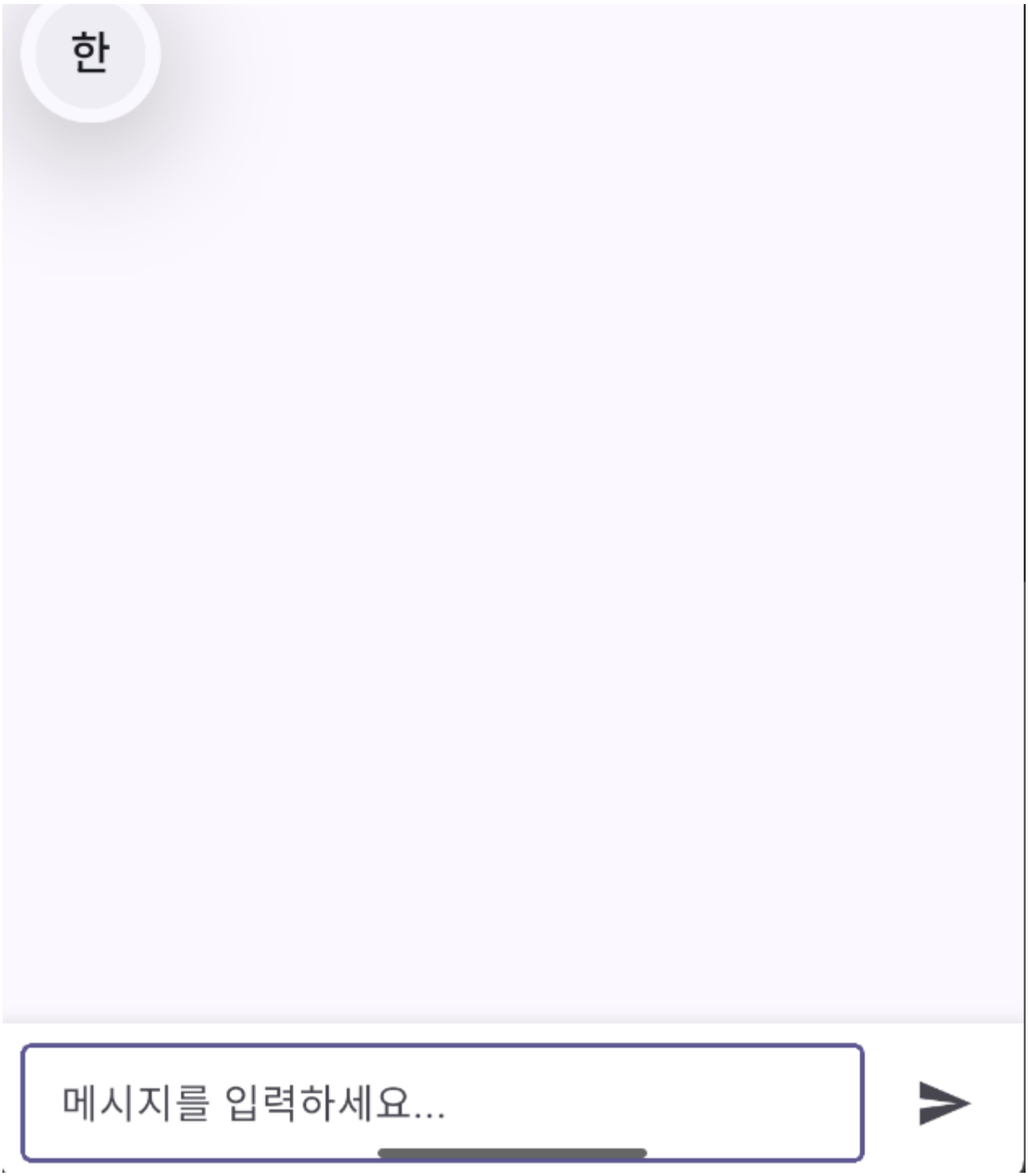
**데이터 동기화:**

- 채팅 화면에서 메시지를 보내면 채팅 목록의 마지막 메시지가 자동으로 업데이트
- `Navigator.push`의 결과를 받아서 `_updateLastMessage`로 상태 업데이트

---

5.5 채팅 화면 (ChatScreen)






**기능 명세:**

- 1:1 실시간 채팅 기능
- 카카오톡 스타일의 메시지 말풍선
- 메시지 전송 시간 표시
- 자동 스크롤 기능

**구성 요소:**

| 위젯 | 명칭 | 설명 |
|---|---|---|
| `StreamBuilder` | 메시지 스트림 | Firestore의 실시간 메시지 구독 |
| `ListView.builder` | 메시지 목록 | 메시지를 역순으로 표시 (최신 메시지가 하단) |

| 위젯 | 명칭 | 설명 |
|---|---|---|
| `MessageBubble` | 메시지 말풍선 | 내 메시지: 오른쪽, 보라색 배경 / 상대 메시지: 왼쪽, 회색 배경 |
| `CircleAvatar` | 상대 프로필 이미지 | 상대 메시지 왼쪽에 표시 |
| `TextField` | 메시지 입력창 | 하단 고정, 둥근 모서리 |
| `IconButton` | 전송 버튼 | 메시지 전송, 전송 중에는 CircularProgressIndicator 표시 |
| `ScrollController` | 스크롤 컨트롤러 | 메시지 전송 시 자동으로 맨 아래로 스크롤 |
| `PopScope` | 뒤로가기 처리 | 메시지 전송 시 채팅 목록에 결과 반환 |

**Firebase 연동 기능:**

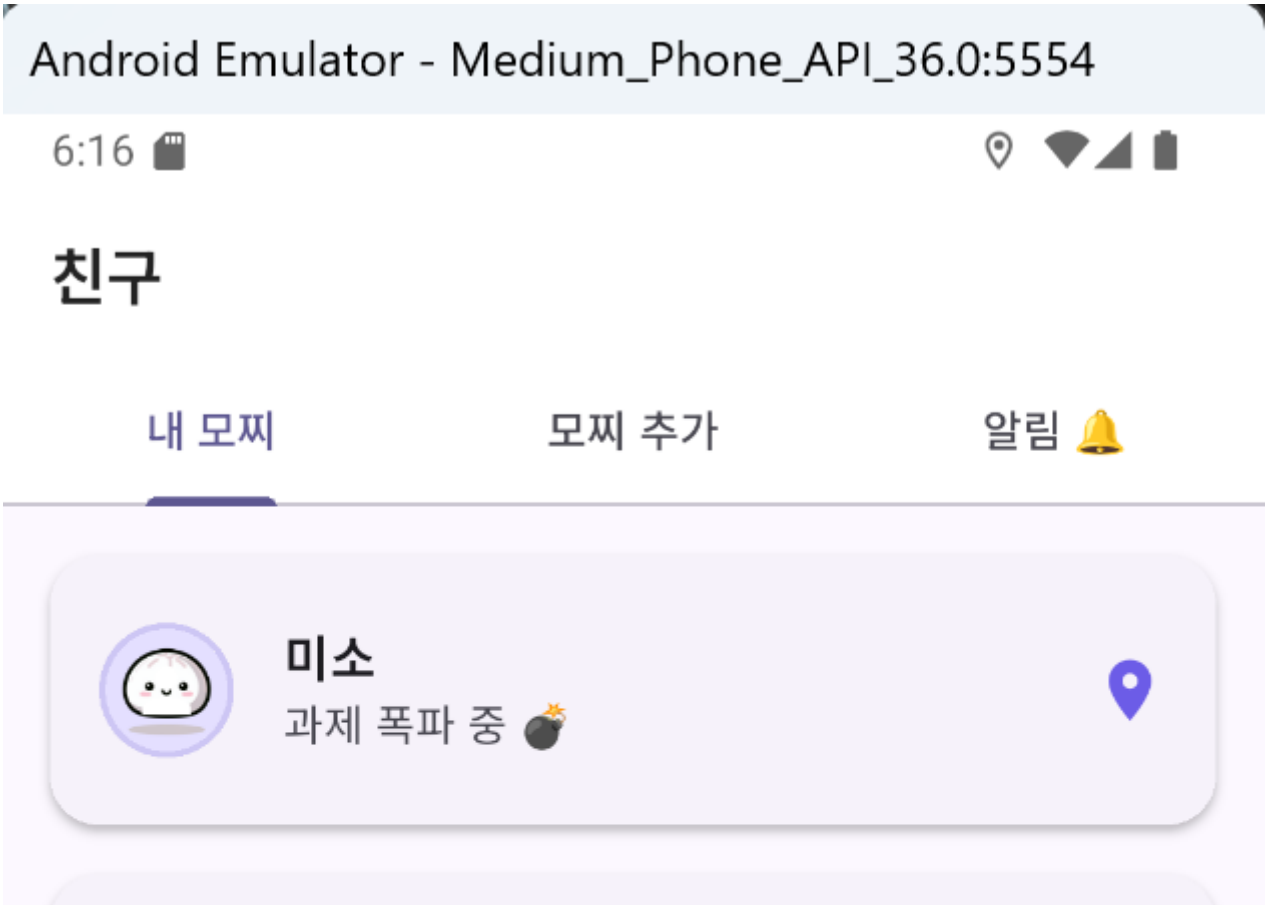
- `ChatService.getMessagesStream()`: 실시간 메시지 스트림 구독
- `ChatService.sendMessage()`: 메시지 전송 및 채팅방 정보 업데이트
- 채팅방 ID는 두 사용자 UID를 정렬하여 생성 (항상 동일한 방 ID)

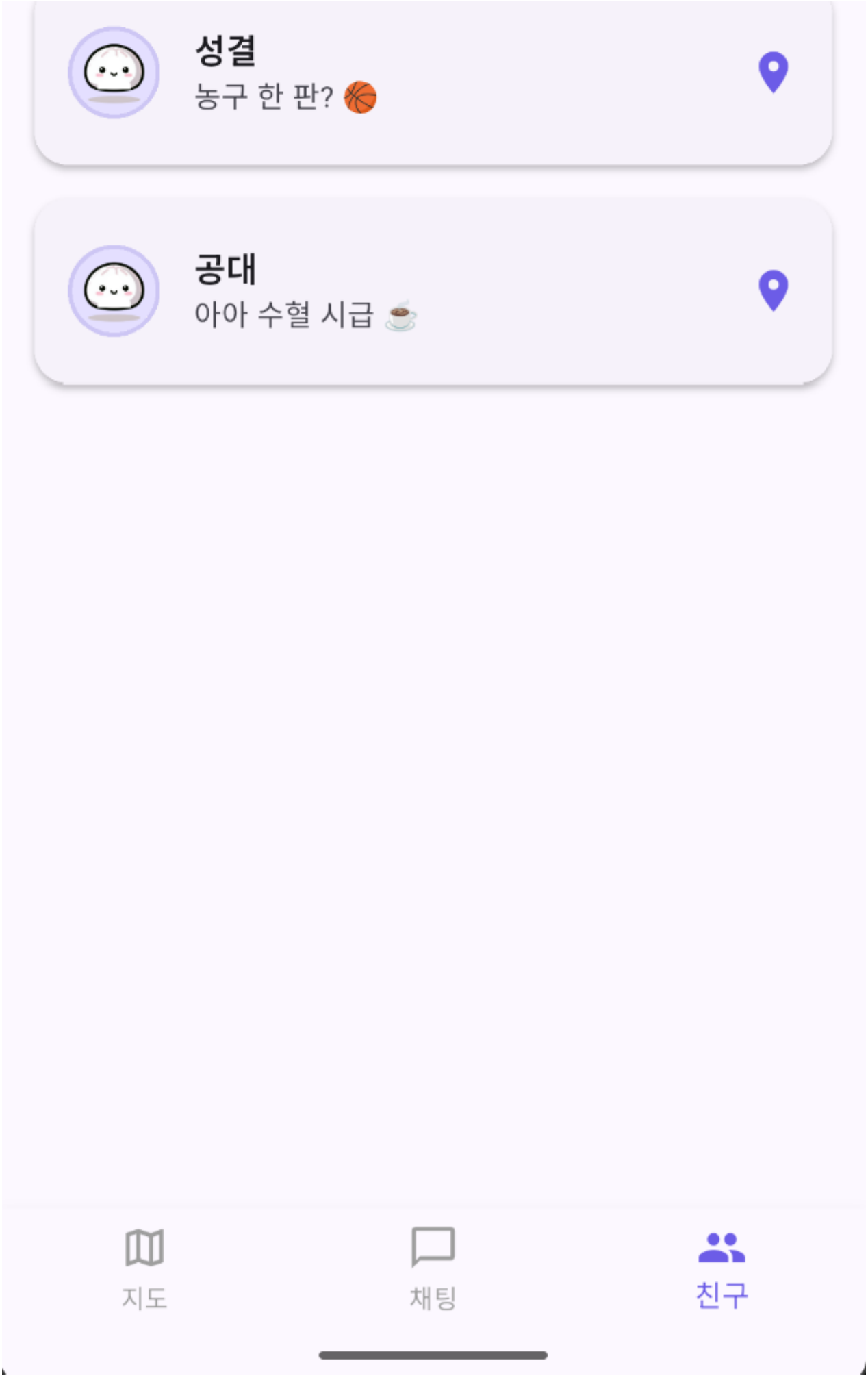
**데모 모드:**

- 데모 친구와의 채팅은 로컬 상태로 관리 (`_demoMessages`)
- 메시지 전송 시 즉시 화면에 표시
- 채팅 목록과 동기화되어 마지막 메시지 업데이트

---

5.6 친구 목록 화면 (MyFriendsTab)




성결
농구 한 판?
공대
아아 수혈 시급
지도
채팅
친구

**기능 명세:**

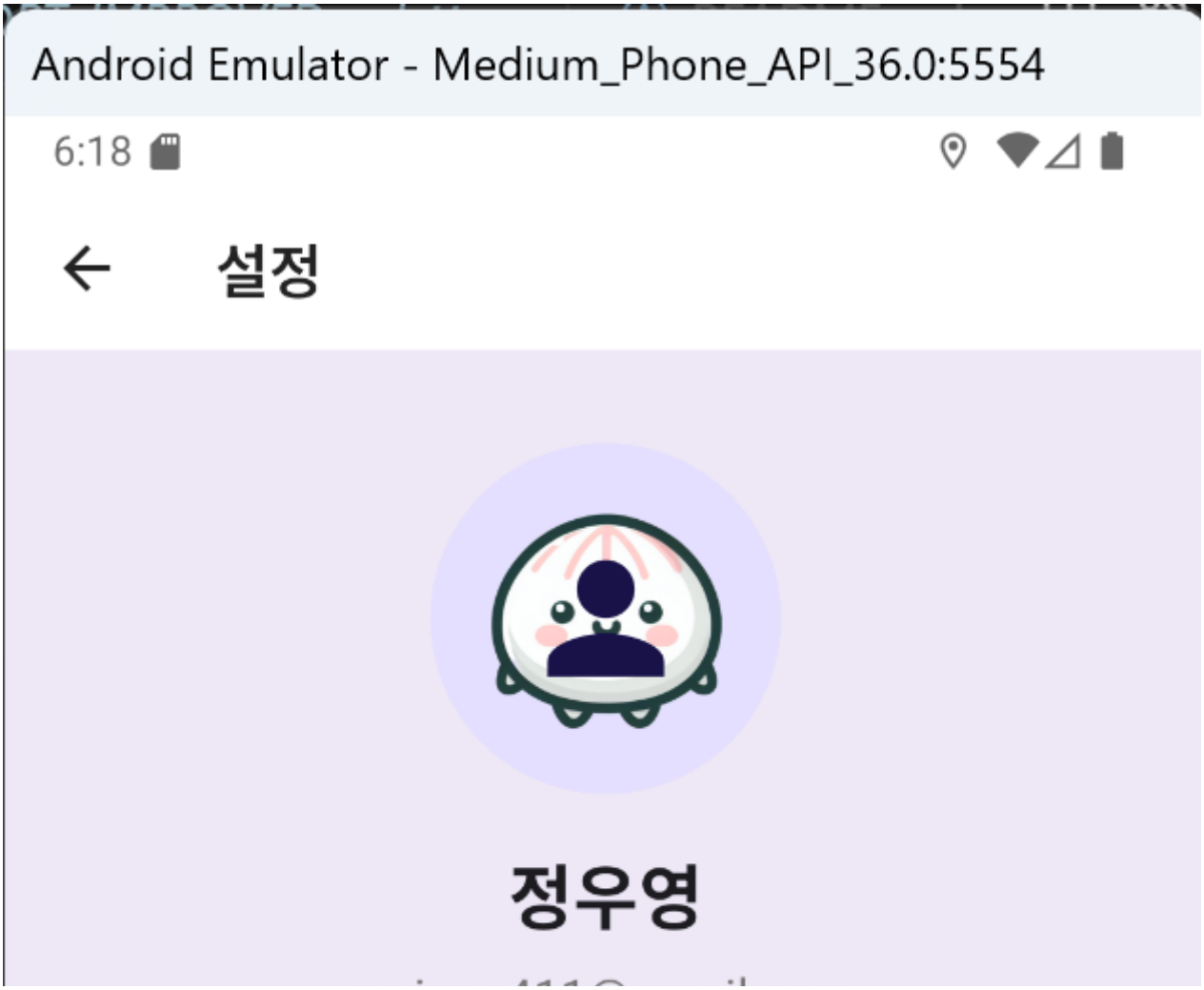
- 추가한 친구 목록을 카드 형태로 표시
- 각 친구의 이름, 상태 메시지, 위치 아이콘 표시
- 친구 클릭 시 해당 친구와의 채팅 화면으로 이동

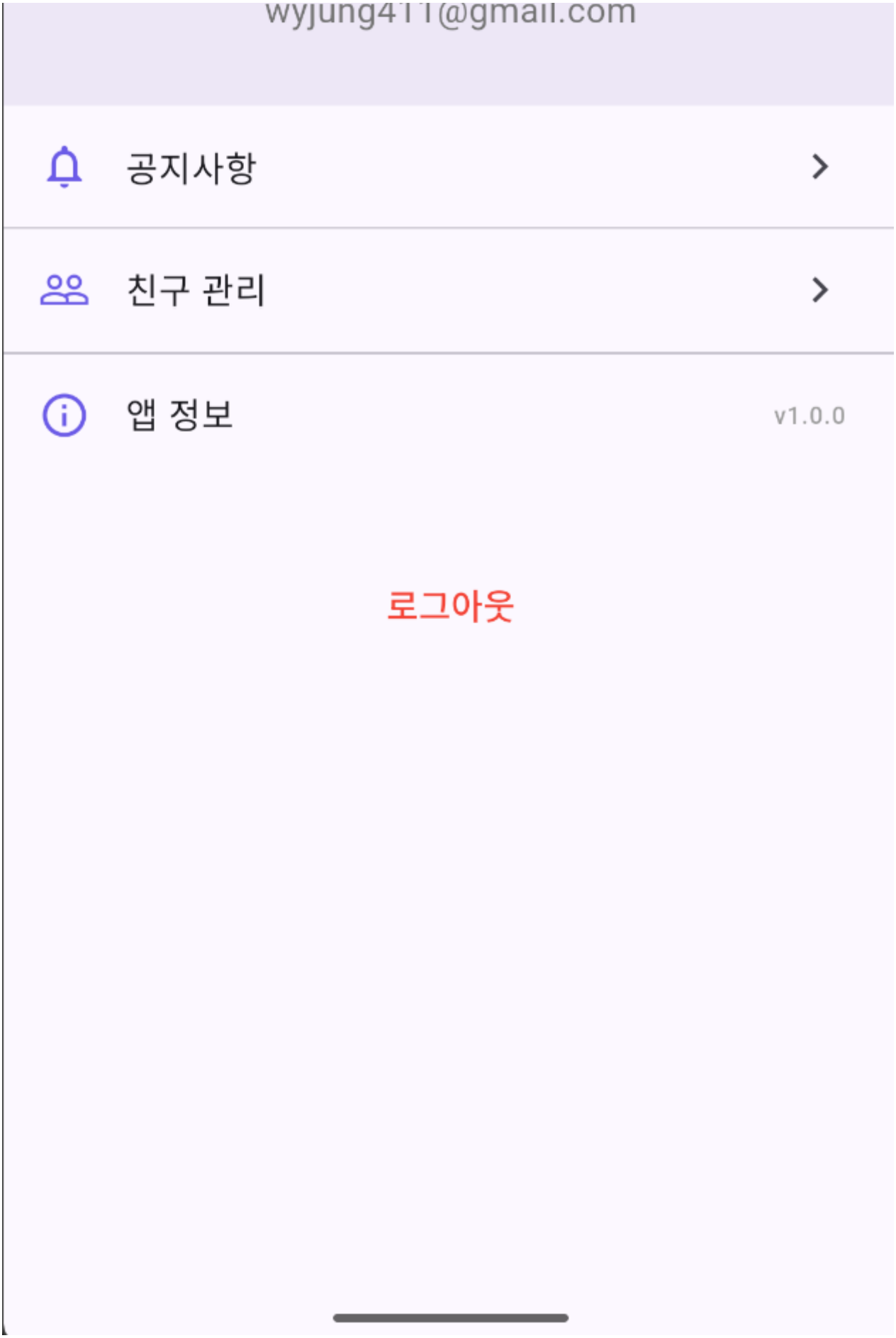
**구성 요소:**

| 위젯 | 명칭 | 설명 |
|---|---|---|
| `ListView.builder` | 친구 리스트 | 친구 목록을 동적으로 생성 |
| `Card` | 친구 카드 | 둥근 모서리 (borderRadius: 16), 여백 적용 |
| `ListTile` | 친구 아이템 | 프로필 이미지, 이름, 상태, 위치 아이콘 포함 |
| `CircleAvatar` | 프로필 이미지 | `marker_friend.png` 이미지 사용, 보라색 테두리 |
| `Text` | 친구 이름 | 굵은 글씨 (fontWeight: w600) |
| `Text` | 상태 메시지 | 친구의 현재 상태 (예: "과제 폭파 중 💣") |
| `Icon` | 위치 아이콘 | 보라색 위치 아이콘, "위치 공유 중" 의미 |
| `EmptyStateWidget` | 빈 상태 위젯 | 친구가 없을 때 안내 메시지 표시 |

---

5.7 마이페이지/설정 화면 (ProfileScreen)






**기능 명세:**

- 사용자 프로필 정보 표시 (이름, 이메일, 프로필 이미지)

- 설정 메뉴 (공지사항, 친구 관리, 앱 정보)
- 로그아웃 기능

**구성 요소:**

| 위젯 | 명칭 | 설명 |
|---|---|---|
| `Container` | 프로필 헤더 | 보라색 반투명 배경, 프로필 정보 영역 |
| `CircleAvatar` | 프로필 이미지 | `profile_placeholder.png` 이미지 사용 (radius: 60) |
| `Text` | 사용자 이름 | 큰 굵은 글씨 (fontSize: 24, fontWeight: bold) |
| `Text` | 이메일 | 회색 작은 글씨 (fontSize: 16) |
| `ListTile` | 공지사항 메뉴 | 알림 아이콘, 보라색 테마 |
| `ListTile` | 친구 관리 메뉴 | 사람 아이콘, 보라색 테마 |
| `ListTile` | 앱 정보 메뉴 | 정보 아이콘, 버전 번호 (v1.0.0) 표시 |
| `TextButton` | 로그아웃 버튼 | 빨간색 텍스트, 중앙 정렬, 확인 다이얼로그 표시 |
| `AlertDialog` | 로그아웃 확인 다이얼로그 | 로그아웃 전 확인 메시지 |

**Firebase 연동 기능:**

- `AuthService.signOut()`: Firebase 로그아웃 실행
- 로그아웃 후 `AuthGate`가 자동으로 로그인 화면으로 리디렉션

---

# 6. 핵심 코드

## 6.1 위치 추적 로직 (LocationProvider)

**목적:** GPS를 사용하여 사용자의 실시간 위치를 추적하고, Firestore에 업데이트하며, AI를 통해 상태를 추론하는 핵심 로직입니다.

```
/// 위치 추적을 초기화합니다.
Future<void> _initializeLocation() async {
  try {
    // 1. 위치 서비스가 활성화되어 있는지 확인
    bool serviceEnabled = await Geolocator.isLocationServiceEnabled();
    if (!serviceEnabled) {
      print('GPS 서비스가 활성화되지 않았습니다.');
      return;
    }

    // 2. 위치 권한이 있는지 확인
    LocationPermission permission = await Geolocator.checkPermission();
    if (permission == LocationPermission.denied) {
      // 3. 권한이 없으면 요청
      permission = await Geolocator.requestPermission();
      if (permission == LocationPermission.denied ||
```

```
          permission == LocationPermission.deniedForever) {
        print('위치 권한이 거부되었습니다.');
        return;
      }
    }

    // 4. 현재 위치 가져오기
    Position position = await Geolocator.getCurrentPosition(
      locationSettings: const LocationSettings(
        accuracy: LocationAccuracy.high,
      ),
    );
    _currentLocation = position;
    _lastAILocation = position;

    // 초기 위치 로드 시 AI 상태 추론
    await _updateStatusWithAI(position);

    notifyListeners();

    // 5. 실시간 위치 감지 스트림 구독
    _positionSubscription = Geolocator.getPositionStream(
      locationSettings: const LocationSettings(
        accuracy: LocationAccuracy.high,
        distanceFilter: 10, // 10미터 이상 이동 시에만 업데이트
      ),
    ).listen(
      (Position currentPosition) {
        _currentLocation = currentPosition;
        notifyListeners();

        // DB에 내 실시간 위치 업데이트
        _updateLocation(currentPosition);

        // 일정 거리 이동 시 AI 상태 추론
        _checkAndUpdateStatusWithAI(currentPosition);
      },
      onError: (error) {
        print('Error in location stream: $error');
      },
    );
  } catch (e) {
    print('Error getting location: $e');
  }
}
```

**단계별 설명:**

1. **위치 서비스 확인**: GPS가 활성화되어 있는지 확인
2. **권한 확인 및 요청**: 위치 권한이 없으면 사용자에게 요청
3. **초기 위치 가져오기**: 앱 실행 시 현재 위치를 한 번 가져옴
4. **AI 상태 추론**: 초기 위치 로드 시 AI를 통해 상태 메시지 생성

5. **실시간 스트림 구독**: 위치 변경 시마다 자동으로 업데이트 (10미터 이상 이동 시에만)

---

## 6.2 AI 상태 추론 로직 (GeminiService)

**목적:** Google Gemini AI를 사용하여 사용자의 위치와 시간 정보를 바탕으로 현재 상황에 맞는 상태 메시지를 생성합니다.

```
/// 현재 위치와 시간을 기반으로 사용자의 활동을 추론합니다.
Future<String> guessActivity(String address, DateTime time) async {
  try {
    // 1. GenerativeModel 초기화 (gemini-pro 모델 사용)
    final model = GenerativeModel(model: 'gemini-pro', apiKey: apiKey);

    // 2. 프롬프트 구성 (강화된 버전)
    final prompt = '''
너는 한국인들이 일상에서 자주 쓰는 센스 있고 유쾌한 상태 메시지 작성기야.

입력 정보:
- 위치: $address
- 시간: ${time.hour}시

위 정보를 바탕으로 현재 상황을 추측해서 상태 메시지를 작성해줘.

**중요 규칙:**
1. 너무 딱딱한 표현("업무 중", "이동 중")은 지양해.
2. 너무 어린 표현("수업 째기")도 지양해.
3. **누구나 공감할 수 있는 위트 있는 표현**을 사용해.
4. **형식:** "한글 3~6글자 + 이모지 1개" (딱 이 포맷만 출력할 것)

**상황별 예시 (참고용):**
- 회사/오피스 + 낮 -> "열일 중 🔥" or "월급 루팡 🤫"
- 식당/번화가 -> "맛집 탐방 😋" or "배에 기름칠 🛢️"
- 공원/산책로 -> "광합성 중 🌿" or "힐링 타임 ✨"
- 집 + 밤/주말 -> "이불 밖 위험 🛌" or "넷플릭스 🍿"
- 카페 -> "커피 수혈 ☕" or "여유 만끽 🍰"
- 헬스장 -> "득근 득근 💪"
- 정체불명/길거리 -> "동에 번쩍 ⚡"
- 학교 + 낮 -> "수업 듣는 중 🏫" or "자체 공강 🤫"
- 학교 + 밤 -> "과제 노예 🧟"

위치와 시간을 분석해서 센스 있는 상태 메시지를 만들어줘. 설명 없이 "한글 3~6글자 + 이모지 1개" 형식으로만 출력해.
''';

    // 3. AI 모델에 프롬프트 전송 및 응답 받기
    final response = await model.generateContent([Content.text(prompt)]);

    // 4. 응답 텍스트 추출 및 정제
    final result = response.text?.trim() ?? '활동 중 🏃';
    String cleanResult = result;
```

```
      // 따옴표로 감싸진 부분이 있으면 추출
      final quoteMatch = RegExp(r"'([^']+)'").firstMatch(result);
      if (quoteMatch != null) {
        cleanResult = quoteMatch.group(1) ?? cleanResult;
      }

      // 불필요한 문장 제거
      cleanResult = cleanResult
          .replaceAll(RegExp(r'입니다\.?'), '')
          .replaceAll(RegExp(r'일 것 같습니다\.?'), '')
          .replaceAll(RegExp(r'이 사람은 '), '')
          .trim();

      // 최종 결과가 비어있거나 너무 길면 기본값 반환
      if (cleanResult.isEmpty || cleanResult.length > 15) {
        return '활동 중 ⛹';
      }

      return cleanResult;
    } catch (e) {
      print('Error getting AI activity guess: $e');
      return '활동 중 ⛹';
    }
  }
```

**단계별 설명:**

1. **모델 초기화**: Google Gemini Pro 모델을 사용하여 AI 인스턴스 생성
2. **프롬프트 구성**: 위치와 시간 정보를 포함한 상세한 프롬프트 작성
3. **AI 호출**: 프롬프트를 AI 모델에 전송하고 응답 받기
4. **결과 정제**: AI 응답에서 불필요한 문장 제거 및 형식 통일
5. **기본값 처리**: 오류 발생 시 기본 상태 메시지 반환

---

## 6.3 실시간 채팅 로직 (ChatService)

**목적:** Firestore를 사용하여 1:1 실시간 채팅 기능을 구현합니다.

```
/// 메시지를 전송하고 채팅방 정보를 업데이트합니다.
Future<void> sendMessage(String friendUid, String message) async {
  try {
    // 1. 현재 로그인한 유저의 uid를 가져옴
    final String? currentUserUid = AuthService.currentUser?.uid;
    if (currentUserUid == null) {
      throw Exception('로그인이 필요합니다.');
    }

    // 2. 채팅방 ID 생성 (두 UID를 정렬하여 항상 같은 방 ID 생성)
    final roomId = getChatRoomId(currentUserUid, friendUid);

    // 3. chat_rooms/{roomId}/messages 컬렉션에 메시지 추가
```

```
      final messagesCollection = _firestore
          .collection('chat_rooms')
          .doc(roomId)
          .collection('messages');

      await messagesCollection.add({
        'senderId': currentUserUid,
        'text': message,
        'timestamp': FieldValue.serverTimestamp(),
      });

      // 4. chat_rooms/{roomId} 문서에 채팅방 정보 업데이트
      final roomDoc = _firestore.collection('chat_rooms').doc(roomId);
      await roomDoc.set({
        'lastMessage': message,
        'lastTime': FieldValue.serverTimestamp(),
        'participants': [currentUserUid, friendUid], // 두 사용자 모두 포함
      }, SetOptions(merge: true));
    } catch (e) {
      print('Error sending message: $e');
      rethrow;
    }
  }

  /// 특정 친구와의 채팅 메시지를 실시간으로 구독하는 스트림을 반환합니다.
  Stream<QuerySnapshot> getMessagesStream(String friendUid) {
    try {
      final String? currentUserUid = AuthService.currentUser?.uid;
      if (currentUserUid == null) {
        return const Stream.empty();
      }

      // 채팅방 ID 생성
      final roomId = getChatRoomId(currentUserUid, friendUid);

      // chat_rooms/{roomId}/messages 컬렉션을 참조
      final messagesCollection = _firestore
          .collection('chat_rooms')
          .doc(roomId)
          .collection('messages');

      // 최신 메시지 순으로 정렬하여 실시간 스트림 반환
      return messagesCollection
          .orderBy('timestamp', descending: true)
          .snapshots();
    } catch (e) {
      print('Error getting messages stream: $e');
      return const Stream.empty();
    }
  }
```

**단계별 설명:**

1. **사용자 인증 확인**: 현재 로그인한 사용자의 UID 확인
2. **채팅방 ID 생성**: 두 사용자 UID를 정렬하여 항상 같은 방 ID 생성
3. **메시지 저장**: `chat_rooms/{roomId}/messages` 컬렉션에 메시지 추가
4. **채팅방 정보 업데이트**: 마지막 메시지와 시간을 채팅방 문서에 업데이트
5. **실시간 스트림**: `snapshots()`를 사용하여 메시지 변경 시 자동으로 UI 업데이트

---

### 6.4 커스텀 마커 생성 로직 (image_utils.dart)

**목적:** 에셋 이미지를 Google Maps 마커로 사용할 수 있도록 변환합니다.

```dart
/// 에셋 이미지를 BitmapDescriptor로 변환하는 함수입니다.
Future<BitmapDescriptor> getBytesFromAsset(
  String assetPath,
  int width,
) async {
  // 1. 에셋 이미지를 ByteData로 로드
  final ByteData data = await rootBundle.load(assetPath);

  // 2. ByteData를 Uint8List로 변환
  final Uint8List bytes = data.buffer.asUint8List();

  // 3. 이미지를 디코딩 (원하는 너비로 리사이즈)
  final ui.Codec codec = await ui.instantiateImageCodec(
    bytes,
    targetWidth: width,
  );

  // 4. 프레임을 가져와서 이미지로 변환
  final ui.FrameInfo frameInfo = await codec.getNextFrame();
  final ui.Image image = frameInfo.image;

  // 5. 이미지를 ByteData로 변환
  final ByteData? imageData = await image.toByteData(
    format: ui.ImageByteFormat.png,
  );

  if (imageData == null) {
    throw Exception('이미지를 ByteData로 변환하는데 실패했습니다.');
  }

  // 6. BitmapDescriptor 생성
  return BitmapDescriptor.fromBytes(imageData.buffer.asUint8List());
}
```

**단계별 설명:**

1. **에셋 로드**: `rootBundle.load()`를 사용하여 에셋 이미지를 ByteData로 로드
2. **바이트 변환**: ByteData를 Uint8List로 변환
3. **이미지 디코딩**: 원하는 너비로 리사이즈하면서 이미지 디코딩

4. **이미지 추출**: 프레임 정보에서 실제 이미지 추출
5. **PNG 변환**: 이미지를 PNG 형식의 ByteData로 변환
6. **마커 생성**: BitmapDescriptor를 생성하여 Google Maps 마커로 사용

---

# 7. 개인 구현 부분

본 프로젝트는 **개인 개발**로 진행되었으며, 모든 기능을 직접 구현했습니다.

## 7.1 구현 범위

- **기획 및 설계**: 100%
- **프론트엔드 개발**: 100%
- **백엔드 설계 (Firestore)**: 100%
- **AI 연동**: 100%
- **UI/UX 디자인**: 100%
- **테스트 및 디버깅**: 100%

## 7.2 주요 구현 내용

1. **위치 추적 시스템**: `geolocator` 패키지를 사용한 실시간 GPS 위치 추적 및 Provider 패턴을 통한 상태 관리
2. **AI 상태 추론**: Google Gemini AI를 활용한 위치 및 시간 기반 상태 메시지 자동 생성
3. **실시간 채팅**: Firestore의 `snapshots()`를 사용한 실시간 메시지 송수신
4. **커스텀 마커**: 에셋 이미지를 Google Maps 마커로 변환하는 유틸리티 함수 구현
5. **소셜 로그인**: Google 및 Kakao 소셜 로그인 연동
6. **UI/UX 디자인**: Material Design 3 기반의 현대적이고 일관된 디자인 시스템 구축

---

# 8. 차별점

## 8.1 AI 기반 자동 상태 추론

대부분의 위치 공유 앱은 사용자가 직접 상태를 입력해야 합니다. **모찌**는 Google Gemini AI를 활용하여 사용자의 위치와 시간 정보를 자동으로 분석하여 상황에 맞는 상태 메시지를 생성합니다.

**예시:**

- 회사 근처 + 오후 2시 → "열일 중 🔥"
- 카페 근처 + 오후 3시 → "커피 수혈 ☕"
- 집 근처 + 밤 10시 → "이불 밖 위험 🛏️"

## 8.2 실시간 위치 공유와 상태 메시지 통합

기존 앱들은 위치만 표시하거나, 상태만 표시하는 경우가 많습니다. **모찌**는 지도에서 친구의 위치와 현재 상태를 동시에 확인할 수 있어, "친구가 지금 어디서 무엇을 하고 있는지"를 한눈에 파악할 수 있습니다.

## 8.3 직관적인 UI/UX

- **카카오톡 스타일 채팅**: 사용자에게 친숙한 메시지 말풍선 디자인

- **커스텀 마커**: 귀여운 모찌 이미지를 사용한 마커로 시각적 재미 제공
- **Material Design 3**: 최신 디자인 가이드라인을 따르는 세련된 UI
- **일관된 브랜딩**: 모찌 퍼플 (#6C5CE7) 컬러를 앱 전체에 통일 적용

### 8.4 배터리 최적화

위치 추적 시 10미터 이상 이동했을 때만 업데이트하도록 설정하여 배터리 소모를 최소화했습니다. 또한 AI 상태 추론은 100미터 이상 이동하거나 5분 이상 경과했을 때만 호출하여 불필요한 API 호출을 줄였습니다.

### 8.5 데모 모드 지원

시연 영상 촬영을 위해 실제 Firebase 데이터와 분리된 데모 모드를 지원합니다. 이를 통해 일관된 시연 환경을 제공할 수 있습니다.

---

## 9. 향후 개선 방향 및 느낀점

### 9.1 향후 개선 방향

**단기 개선 사항**

1. **푸시 알림 강화**
    - 친구 요청 알림
    - 새 메시지 알림
    - 친구 위치 변경 알림 (선택적)
2. **프로필 이미지 업로드**
    - Firebase Storage를 사용한 이미지 업로드
    - 프로필 이미지 편집 기능
3. **그룹 채팅**
    - 여러 명과 동시에 채팅할 수 있는 그룹 채팅방
    - 그룹 초대 및 관리 기능
4. **위치 히스토리**
    - 친구의 위치 이동 경로 표시
    - 특정 시간대 위치 조회

**중장기 개선 사항**

1. **AI 기능 강화**
    - 대화 내용 분석을 통한 상태 추론
    - 위치 패턴 학습을 통한 예측 기능
    - 개인화된 상태 메시지 추천
2. **소셜 기능 확장**

- 친구 그룹 생성 및 관리
- 위치 기반 이벤트 생성
- 공개/비공개 위치 공유 설정

3. **성능 최적화**
   - 이미지 캐싱 및 압축
   - 리스트 가상화 (ListView.builder 최적화)
   - 오프라인 모드 지원

4. **보안 강화**
   - 위치 정보 암호화
   - 친구 요청 승인 프로세스 강화
   - 개인정보 보호 설정

### 9.2 느낀점

이번 Flutter를 이용한 앱 개발 경험은 프로젝트를 통해 더욱 체계적으로 쌓을 수 있었습니다. 특히 다음과 같은 부분에서 많은 것을 배울 수 있었습니다:

1. **상태 관리의 중요성**: Provider 패턴을 도입하여 비즈니스 로직과 UI 로직을 분리한 것이 코드의 가독성과 유지보수성을 크게 향상시켰습니다.

2. **Firebase 연동의 편리함**: Firestore의 실시간 스트림 기능을 활용하여 채팅 기능을 구현하면서, 서버 없이도 실시간 데이터 동기화가 가능하다는 것을 경험했습니다.

3. **AI 연동의 흥미로움**: Google Gemini AI를 활용하여 사용자의 상황을 자동으로 추론하는 기능을 구현하면서, AI가 단순한 도구를 넘어 사용자 경험을 향상시키는 핵심 요소가 될 수 있다는 것을 깨달았습니다.

4. **UI/UX 디자인의 중요성**: Material Design 3 가이드라인을 따르고 일관된 브랜딩을 적용한 것이 앱의 완성도를 크게 높였습니다.

5. **문제 해결 능력**: Firebase 초기 설정, 패키지 충돌, 상태 관리 등 다양한 문제를 해결하면서 디버깅 능력과 문제 해결 능력이 한 단계 성장할 수 있었습니다.

특히, 기능을 설계할 때 사용자 흐름을 고려하여 "스플래시 → 로그인 → 지도 → 채팅" 과정을 자연스럽게 이어지도록 구현한 부분은 정말 애증의 구현과정이었습니다. 그 부분에서 너무 많은 오류들과 이슈 문제가 발생하여 애를 먹었지만 해결해 나가면서 해결능력이 한 단계 더 성장할 수 있게 된 것 같아 뿌듯했습니다.

이번 프로젝트는 단순한 앱 구현을 넘어서, 기획부터 디자인 반영, 기능 개발, 디버깅, 테스트까지 모든 과정이 포함된 종합적인 실습이었고, 그만큼 실력이 한층 성장할 수 있었던 값진 경험이었습니다. 앞으로의 개발 여정에도 큰 밑거름이 될 것 같습니다.

---

## 부록

### A. 프로젝트 구조

```
lib/
├── main.dart
```

```
├── firebase_options.dart
├── models/
│   ├── user_model.dart
│   └── chat_model.dart
├── screens/
│   ├── splash_screen.dart
│   ├── auth/
│   │   ├── auth_gate.dart
│   │   ├── login_screen.dart
│   │   └── phone_verification_screen.dart
│   ├── map_screen.dart
│   ├── chat/
│   │   └── chat_screen.dart
│   ├── chat_list_screen.dart
│   ├── friends_list_screen.dart
│   ├── tabs/
│   │   └── friends/
│   │       ├── my_friends_tab.dart
│   │       ├── add_friend_tab.dart
│   │       └── friend_requests_tab.dart
│   ├── profile_screen.dart
│   └── main_navigator.dart
├── services/
│   ├── auth_service.dart
│   ├── database_service.dart
│   ├── chat_service.dart
│   ├── gemini_service.dart
│   └── notification_service.dart
├── providers/
│   └── location_provider.dart
├── widgets/
│   ├── empty_state_widget.dart
│   ├── auth/
│   │   └── social_login_button.dart
│   └── chat/
│       └── message_bubble.dart
└── utils/
    ├── image_utils.dart
    └── marker_generator.dart
```

## B. 주요 패키지 목록

```yaml
dependencies:
  flutter:
    sdk: flutter
  firebase_core: ^3.3.0
  firebase_auth: ^5.1.2
  cloud_firestore: ^5.1.0
  google_sign_in: ^6.2.1
  kakao_flutter_sdk_user: ^1.9.0
  google_maps_flutter: ^2.6.1
```

```
  geolocator: ^14.0.2
  geocoding: ^4.0.0
  google_generative_ai: ^0.4.4
  provider: ^6.1.2
  intl: ^0.20.2
  flutter_contacts: ^1.1.8
  firebase_messaging: ^15.1.3
  flutter_local_notifications: ^17.2.3
```

### C. Firestore 데이터베이스 구조

```
users/
  {uid}/
    - email: String
    - displayName: String
    - phoneNumber: String?
    - createdAt: Timestamp

locations/
  {uid}/
    - position: GeoPoint
    - status: String
    - updatedAt: Timestamp

chat_rooms/
  {roomId}/
    - lastMessage: String
    - lastTime: Timestamp
    - participants: [String]
    messages/
      {messageId}/
        - senderId: String
        - text: String
        - timestamp: Timestamp

friend_requests/
  {requestId}/
    - fromUid: String
    - toUid: String
    - status: "pending" | "accepted" | "rejected"
    - createdAt: Timestamp
```

### D. 디자인 정의

**메인 컬러:**

- Primary: `#6C5CE7` (모찌 퍼플)
- Secondary: `#A29BFE` (연한 보라색)
- Tertiary: `#6C5CE7` (동일)

**테마:**

- Material Design 3 기반
- AppBar: 흰색 배경, 검은색 텍스트, elevation: 0
- BottomNavigationBar: 선택 메인 컬러, 비선택 회색
- Card: borderRadius: 16, elevation: 2
- Button: borderRadius: 12, 메인 컬러 배경

---

**보고서 작성 완료**

*본 보고서는 모찌 앱 개발 프로젝트의 전체 과정과 결과를 정리한 문서입니다.*